

Practical cryptanalysis of pseudorandom correlation generators based on quasi-Abelian syndrome decoding

Charles Bouillaguet¹[0000–0001–9416–6244],
Claire Delaplace²[0000–0002–5314–1806],
Mickaël Hamdad^{1,2}[0009–0003–0597–6221], and
Damien Vergnaud¹[0000–0002–2113–3967]

¹ Sorbonne Université, CNRS, LIP6, F-75005 Paris, France
{charles.bouillaguet,mickael.hamdad,damien.vergnaud}@lip6.fr

² Laboratoire MIS, Université de Picardie Jules Verne, France
claire.delaplace@u-picardie.fr

Abstract. Quasi-Abelian Syndrome Decoding (QA-SD) was introduced by Bombar et al (Crypto 2023) in order to obtain pseudorandom correlation generators for Beaver triples over small fields. This theoretical work was turned into a concrete and efficient protocol called F4OLEage by Bombar et al. (Asiacrypt 2024) that allows several parties to generate Beaver triples over $\text{GF}(2)$.

We propose efficient algorithms to solve the decoding problem underlying the QA-SD assumption. We observe that it reduces to a sparse multivariate polynomial interpolation problem over a small finite field where the adversary only has access to random evaluation points, a blind spot in the otherwise rich landscape of sparse multivariate interpolation. We develop new algorithms for this problem: using simple techniques we interpolate polynomials with up to two monomials. By sending the problem to the field of complex numbers and using convex optimization techniques inspired by the field of “compressed sensing”, we can interpolate polynomials with more terms.

This enables us to break in practice parameters proposed by Bombar et al. at Crypto’23 and Asiacrypt’24 as well as Li et al. at Eurocrypt’25 (IACR flagship conferences Grand Slam). In the case of the F4OLEage protocol, our implementation distinguishes the output of the pseudorandom correlation generator from random with advantage 60% in a few hours. This not only invalidates the security proofs, but it yields real-life privacy attacks against multiparty protocols using the Beaver triples generated by the broken pseudorandom correlation generators.

Keywords. Quasi-Abelian Syndrome Decoding. Sparse polynomial interpolation. Practical attacks.

1 Introduction

Pseudorandom correlation generators. A *pseudorandom correlation generator* (PCG) is a cryptographic primitive that allows two or more parties to locally

generate correlated random variables from short shared randomness, without further communication, such that the outputs are indistinguishable from truly random correlated variables by any efficient adversary. PCGs have recently found numerous applications in secure multiparty computation and related protocols, where they enable the efficient, non-interactive generation of large volumes of correlated randomness, such as multiplication triples (also known as Beaver triples) or *Oblivious Linear Evaluation* (OLE) correlations where two parties P_0 and P_1 respectively receive shares (x_0, z_0) and (x_1, z_1) such that $x_0 x_1 = z_0 + z_1$.

In a standard PCG, the output correlation is determined entirely by the shared seed. In contrast, a *programmable* PCG allows a party to specify certain parts of the output (e.g., to “reuse” a piece of his input in multiple instances), and the generator produces the other side so that the desired correlation still holds and the output remains indistinguishable from a truly random correlated sample. This property allows notably to generate multiparty correlations from pairwise correlations.

Bombar et al. introduced in [7] programmable PCGs for OLE over arbitrary finite fields $\mathbb{F}_q$ with $q > 2$. Building on this, the $\mathbb{F}_4$ OLEAGE protocol [6] proposed an efficient PCG for multiplication triples over $\mathbb{F}_4$ and $\mathbb{F}_2$. Li et al. [22] extended these techniques to arbitrary finite fields, introducing programmable PCGs for both arithmetic and *Boolean authenticated triples*, with *sublinear communication* and *quasi-linear computation*. The security of these PCGs relies on a new assumption called the *Quasi-Abelian Syndrome Decoding* problem.

1.1 The Quasi-Abelian Syndrome Decoding problem

The Quasi-Abelian Syndrome Decoding (QA-SD) problem was introduced in [7]. We describe here the instances that are of interest to us, that have been explicitly suggested in [7, 6, 22].

Let q be a prime power and write $d := q - 1$. Note that any non-zero element x of the finite field $\mathbb{F}_q$ satisfies $x^d = 1$. Let $\mathcal{R}$ denote the multivariate polynomial quotient ring

$$\mathcal{R} := \mathbb{F}_q[X_1, \dots, X_n] / (X_1^d - 1, \dots, X_n^d - 1). \quad (1)$$

Let ϕ denote the canonical homomorphism from $\mathbb{F}_q[X_1, \dots, X_n]$ to $\mathcal{R}$ (ϕ essentially reduces the exponents of all variables modulo d). An element $f \in \mathcal{R}$ is conveniently represented by $\tilde{f} \in \mathbb{F}_q[X_1, \dots, X_n]$, where $f = \phi(\tilde{f})$. If we additionally enforce that the exponent of each variable in $\tilde{f}$ is less than d , then $\tilde{f}$ is unique. In this paper, we often identify f with $\tilde{f}$ and treat elements of $\mathcal{R}$ as “normal” multivariate polynomials over $\mathbb{F}_q$, for instance by evaluating them on some vector $\mathbf{x} \in \mathbb{F}_q^n$. In any case, it follows from the above discussion that there are d^n distinct monomials and $\mathcal{R}$ can be seen as a vector space of dimension d^n over $\mathbb{F}_q$.

Given a vector $\mathbf{x} \in \mathbb{F}_q^n$, denote by $\phi_{\mathbf{x}}$ the function $f \mapsto \tilde{f}(\mathbf{x})$ that evaluates an element of $\mathcal{R}$ at $\mathbf{x}$. When $\mathbf{x}$ has no zero coordinate, (i.e., when $\mathbf{x} \in (\mathbb{F}_q^*)^n$) then $\phi_{\mathbf{x}}$ is a ring homomorphism. Indeed, if $f, g \in \mathcal{R}$, then it is obvious that

$(\tilde{f} + \tilde{g})(\mathbf{x}) = \tilde{f}(\mathbf{x}) + \tilde{g}(\mathbf{x})$. Write $h := fg \in \mathcal{R}$. By definition of $\mathcal{R}$, there exist multivariate polynomials $r_1, \dots, r_n \in \mathbb{F}_q[X_1, \dots, X_n]$ such that

$$\tilde{h} = \tilde{f}\tilde{g} + \sum_{i=1}^n r_i(X_i^d - 1).$$

When all coordinates of $\mathbf{x}$ are non-zero, they all belong to the multiplicative group $\mathbb{F}_q^*$ of order $q - 1$, and all the “ $X_i^d - 1$ ” polynomials evaluate to zero on $\mathbf{x}$. Therefore $\tilde{h}(\mathbf{x}) = \tilde{f}(\mathbf{x})\tilde{g}(\mathbf{x})$. This is not true if (say) $x_i = 0$, because then we have $\tilde{h}(\mathbf{x}) = \tilde{f}(\mathbf{x})\tilde{g}(\mathbf{x}) - r_i(\mathbf{x})$. This is one of the main motivation behind the introduction of $\mathcal{R}$: a single OLE relation $a + b = uv$ in $\mathcal{R}$ can be “projected” onto d^n OLE relations in $\mathbb{F}_q$ by evaluating it on all the points of $(\mathbb{F}_q^*)^n$. By the Chinese Remainder Theorem, this means that there is in fact a ring homomorphism between $\mathcal{R}$ and $\mathbb{F}_q^{d^n}$ (where multiplication is done in parallel on all coordinates). A consequence is that a polynomial in $\mathcal{R}$ can be represented either by its d^n coefficients in $\mathbb{F}_q$ or by its d^n evaluations on $(\mathbb{F}_q^*)^n$.

A t -sparse element of $\mathcal{R}$ is the sum of t monomials, each of them multiplied by a non-zero scalar. Denote by $\mathcal{R}_t$ the subset of $\mathcal{R}$ formed by t -sparse polynomials.

Definition 1 (Search QA-SD). Let q a prime power (the size of the finite field), $n \geq 1$ an integer (the number of variables), $c \geq 2$ an integer (the “compression factor”) and $t \geq 1$ an integer (the error Hamming weight). Given access to a single pair of the form $((a_1, \dots, a_{c-1}), e_0 + \sum_{i=1}^{c-1} a_i e_i)$, where $(a_1, \dots, a_{c-1}) \xleftarrow{\$} \mathcal{R}^{c-1}$ and $(e_0, \dots, e_{c-1}) \xleftarrow{\$} \mathcal{R}_t^c$, the goal is to recover the “error terms” $e_0, \dots, e_c$.

Definition 2 (Decisional QA-SD). In the same setting, the goal is to distinguish the following distributions:

$$\begin{aligned} \mathcal{D}_0 &: (a_1, \dots, a_{c-1}, u) & \text{where } a_i \xleftarrow{\$} \mathcal{R} \text{ and } u \xleftarrow{\$} \mathcal{R} \\ \mathcal{D}_1 &: (a_1, \dots, a_{c-1}, e_0 + \sum_{i=1}^{c-1} a_i e_i) & \text{where } a_i \xleftarrow{\$} \mathcal{R} \text{ and } e_i \xleftarrow{\$} \mathcal{R}_t \end{aligned}$$

We say that a randomized algorithm $\mathcal{A}$ (τ, ϵ) -solves the decisional QA-SD problem if it runs in less than τ elementary operations and its distinguishing advantage is at least ϵ , namely:

$$\left| \Pr \left(\mathcal{A}(x) = 1 \mid x \xleftarrow{\$} \mathcal{D}_0 \right) - \Pr \left(\mathcal{A}(x) = 1 \mid x \xleftarrow{\$} \mathcal{D}_1 \right) \right| \geq \epsilon$$

Suppose that we have an algorithm $\mathcal{B}$ (a “recognizer”) that when presented with a QA-SD sample returns 1 with probability p and returns 0 with overwhelming probability when presented with a random sample. It can be converted into a distinguisher $\mathcal{A}$ with advantage $p/2$ as follows: run $\mathcal{B}$ on the input; if $\mathcal{B}$ returns 1, then return 1; otherwise return a random bit. Clearly, a “recognizer” with non-negligible probability p yields a distinguisher with non-negligible advantage and invalidates the decisional QA-SD assumption. In the sequel, we focus on such recognizers and discuss the probability that they recognize a QA-SD sample when they are given one.

It is shown in [7] that if an algorithm $\mathcal{A}$ (τ, ϵ) -solves the decisional QA-SD problem, then there exists an algorithm that solves the search QA-SD problem with overwhelming probability and running time $\mathcal{O}(qn^4\tau/\epsilon^2)$. In other terms, the decisional problem is not much easier than the search problem.

Quoting [7], “the QA-SD assumption states that the above decisional problem should be hard (for appropriate parameters)”. Quoting [6]: “the QA-SD $(q, c, t, \mathbb{G})$ assumption holds when this problem is hard for every non-uniform polynomial time distinguisher”. The size of the input to the adversary is $c(q-1)^n$ elements of $\mathbb{F}_q$. We interpret these definitions of the hardness of QA-SD in the following way: c, q and t are fixed, and the only parameter that can grow is n . So the QA-SD assumption holds if no algorithm with running time $\text{poly}((q-1)^n)$ can solve the decisional problem with advantage $1/\text{poly}((q-1)^n)$.

1.2 Our contribution: breaking specific instantiations of QA-SD

Table 1 recapitulates the concrete QA-SD parameters that have been proposed and highlight those that we can attack. We present efficient “recognizers” for QA-SD sample with non-negligible success probability. They can be practical in some cases. We have implemented them (in Python) and executed them on some parameter sets to verify that they actually work in practice. In some cases, we are able to fully solve the search problem in addition to the decisional problem.

Where	q	n	c	t	noise	λ	attack proba.	
[7]	3	25,30,35	$\frac{2}{4}$	$\frac{152}{64}$	regular	128	13.2% (PP) / 42% (T) 31% (T)	
[6]	4	14,16,18	$\frac{3}{4}$	$\frac{27}{4}$	regular	128	61% (PP) 61% (T)	
[22]	4	19	9	9	regular	80	$\perp$ 100% (PP) 43% (P)	
		16,18	2	42	uniform		128	97.8% (P) 34.7% (T)
		19,20	5	17				97.4% (PP) 1.38% (P) – 27% (T)
		19	2	81	regular	128	$\perp$	
		19,21	5	27				
		17	2	66	uniform			
		18,19,20	4	33				
	8	10,12	$\frac{2}{3}$	$\frac{49}{15}$	regular	80	72% (P)	
						128		
	16	7,8	$\frac{6}{9}$			80		
						128		

Table 1. QA-SD parameters proposed in the literature. The column “attack proba.” shows the success probability of QA-SD recognizers that we propose. PP means that we actually executed (parts of) the attack to verify that it is practical. P means that the attack is practical, even if we did not actually try to run it. T means that the attack could be executed in theory, if enough RAM were available. $\perp$ means that our attacks do not apply.

This enabled us to break instances of some proposed parameter sets in practice, notably the “aggressive” parameter set of $\mathbb{F}_4\text{OLEAGE}$ given in [6]. We have two kind of algorithms:

- Simple ones described in Section 2.2 that require almost no memory besides storing the instance of the problem that can be quite large (up to 10G field elements in some cases). We consider that they are practical even if we have not run them to completion.
- A more complex one described in section 3 that has non-trivial memory requirements. We have executed it in practice in some cases, where we consider that the parameters are practically broken. We also know that, if enough RAM were available, it could be executed and successful in some other cases. We then consider that the relevant parameter sets are broken “in theory”.

Table 1 summarizes the proposed parameter sets and display the success probability of our QA-SD recognizers. A significant fraction of the proposed parameters are practically broken.

1.3 Notations

Let $\mathbb{K}$ be a field (finite or not). We denote the multiplicative group of non-zero elements of $\mathbb{K}$ as $\mathbb{K}^*$. We denote by θ a generator of $\mathbb{F}_q^*$. Let $N, c > 0$ be two positive integers. Given two vectors $\mathbf{u}$ and $\mathbf{v}$ from $\mathbb{K}^N$, we denote by $\langle \mathbf{u}, \mathbf{v} \rangle$ and by $\mathbf{u} \otimes \mathbf{v}$ respectively the dot product and the tensor product between $\mathbf{u}$ and $\mathbf{v}$. We extend these two notations to vectors of $\mathcal{R}^c$ (i.e., vectors whose coordinates are polynomials of $\mathcal{R}$), with $\mathcal{R}$ defined as in Equation 1.

We denote by $wt(\mathbf{v})$ the number of non-zero components of a given vector $\mathbf{v} \in \mathbb{K}^N$. If $wt(\mathbf{v}) = t \ll N$, we say that $\mathbf{v}$ is t -sparse. As already stated in Section 1.1 we extend this definition to polynomials. The set of non-zero coefficients (resp., monomials) of a t -sparse vector (resp., polynomial) is called its *support*. Given a polynomial $P \in \mathbb{K}[X_1, \dots, X_n]$, we denote by $\deg_{X_i}(P)$ the degree of P in variable X_i .

Let $\omega \in \mathbb{K}$ be a primitive N -th root of unity in $\mathbb{K}$. We define the *Discrete Fourier Transform* (DFT) as the linear operator that maps an N -tuple $\mathbf{g} = (g(0), \dots, g(N-1))$ to another $\hat{\mathbf{g}} = (\hat{g}(0), \dots, \hat{g}(N-1))$ and also the *Inverse DFT*, such that for all $0 \leq k < N$,

$$\hat{g}(k) = \sum_{j=0}^{N-1} g(j)\omega^{jk}, \quad \text{and} \quad g(k) = \frac{1}{N} \sum_{j=0}^{N-1} \hat{g}(j)\omega^{-jk}. \quad (2)$$

Let $z = u + \mathbf{i}v$ be a complex number. We denote by $\bar{z}$ its complex conjugate. Recall that the norm of z is given by $|z| = \sqrt{u^2 + v^2}$. From here, we denote the ℓ_1 -norm of a complex vector $\mathbf{z} = (z_1, \dots, z_N)$ by

$$\|\mathbf{z}\|_1 = \sum_{j=1}^N |z_j|. \quad (3)$$

2 Attack description

In this section we describe the main ideas of our attack against the QA-SD assumption. It yields efficient distinguishers for decisional QA-SD and efficient algorithms for the search problem. It is worth noting that, even though there is a search-to-decision reduction, the decisional problem can be concretely easier.

2.1 Breaking Cryptographic Protocols using a QA-SD distinguisher

In this subsection, we present a (non-exhaustive) list of simple observations showing that our attacks do not merely invalidate the Decisional QA-SD assumption underlying cryptographic protocols—and hence their security proofs—but also actually break the PCGs themselves. This, in turn, can lead to concrete attacks against advanced cryptographic protocols that rely on them.

A *correlation* $\mathcal{C}$ is a probabilistic polynomial time (PPT) algorithm that, on input a security parameter λ , outputs a pair of correlated strings (σ_0, σ_1) . A PCG for a target correlation $\mathcal{C}$ consists of two algorithms, **Gen** and **Expand**. The generation algorithm $\text{Gen}(1^\lambda, \text{params})$ outputs a pair of (short) correlated seeds $(\text{seed}_0, \text{seed}_1)$, and the expansion algorithm $\text{Expand}(b, \text{seed}_b)$ produces a long string σ_b for $b \in \{0, 1\}$. A PCG should satisfy the following properties:

- *Correctness*: the pair (σ_0, σ_1) is computationally indistinguishable from a sample drawn from $\mathcal{C}(1^\lambda)$.
- *Security*: for any $b \in \{0, 1\}$, given seed_{1-b} , the string σ_b is computationally indistinguishable from a sample drawn from $\mathcal{C}(1^\lambda)$ conditioned on being consistent with $\text{Expand}(1-b, \text{seed}_{1-b})$.

PCGs for OLEs over $\mathbb{F}_q$ for $q > 2$ from [7, 6]. The OLE correlation of length N over a finite field $\mathbb{F}_q$ is defined by the PPT algorithm $\mathcal{C}$ that outputs vectors $\sigma_b = (\mathbf{x}_b, \mathbf{z}_b) \in (\mathbb{F}_q^2)^N$ for $b \in \{0, 1\}$, such that $x_{0,i}x_{1,i} = z_{0,i} + z_{1,i}$ for all $i \in \{1, \dots, N\}$. The blueprint of the PCG family for OLE correlations in [7] is as follows (for $N = d^n$):

- **Gen**($c, \mathcal{R}$): samples a (public) vector $\mathbf{a} \in \mathcal{R}^c$, and, for $b \in \{0, 1\}$, generates a vector $\mathbf{e}_b \in \mathcal{R}_t^c$. It then computes³ additive shares v_0 and v_1 of all coefficients of all polynomials in $\mathbf{e}_0 \otimes \mathbf{e}_1$. It outputs $\text{seed}_b = (\mathbf{a}, \mathbf{e}_b, v_b)$ for $b \in \{0, 1\}$.
- **Expand**(b, seed_b): computes $x_b = \langle \mathbf{a}, \mathbf{e}_b \rangle$ and $z_b = \langle \mathbf{a} \otimes \mathbf{a}, v_b \rangle$.

The outputs (x_b, z_b) of **Expand**(b, seed_b), for $b \in \{0, 1\}$, are elements of $\mathcal{R}^2$ satisfying $x_0x_1 = z_0 + z_1$. Using the ring isomorphism between $\mathcal{R}$ and $\mathbb{F}_q^N$, one obtains vectors pairs $(\mathbf{x}_b, \mathbf{z}_b) \in (\mathbb{F}_q^2)^N$ for $b \in \{0, 1\}$ such that $x_{0,i}x_{1,i} = z_{0,i} + z_{1,i}$ for all $i \in \{1, \dots, N\}$. To ensure security, it must be infeasible for an adversary,

³ This step is optimized using *Distributed Point Functions* (DPFs), a cryptographic primitive that allows parties to share a point function—nonzero at a single input—such that each party holds a key enabling local evaluation, and the value is revealed only when their outputs are combined.

knowing $\mathbf{a}$, to distinguish $x_b \in \mathcal{R}$ (or equivalently $\mathbf{x}_b \in \mathbb{F}_q^N$) from a uniformly random vector in $\mathbb{F}_q^N$, even without access to z_b . This is precisely an instance of the Decisional Quasi-Abelian Syndrome Decoding problem. Consequently, any adversary capable of solving QA-SD breaks the security of the PCG for OLE with the same parameters.

PCGs for multiplication triples from [6]. In [10], Boyle et al. showed how to transform a PCG for OLE into a PCG for multiplication triples. These correspond to a correlation over $\mathbb{F}_q$ that outputs vectors $\sigma_b = (\mathbf{x}_b, \mathbf{y}_b, \mathbf{z}_b) \in (\mathbb{F}_q^3)^{m/2}$ for $b \in \{0, 1\}$, such that $(x_{0,i} + x_{1,i}) \cdot (y_{0,i} + y_{1,i}) = z_{0,i} + z_{1,i}$ for all i . Their construction uses two OLE correlations to generate one multiplication triple via:

$$\begin{aligned} x'_0 &= x_0^{(0)}, & y'_0 &= x_0^{(1)}, & z'_0 &= z_0^{(0)} + z_0^{(1)} + x_0^{(0)} \cdot x_0^{(1)}, \\ x'_1 &= x_1^{(1)}, & y'_1 &= x_1^{(0)}, & z'_1 &= z_1^{(0)} + z_1^{(1)} + x_1^{(0)} \cdot x_1^{(1)}. \end{aligned}$$

It is easy to verify that this satisfies $(x'_0 + x'_1) \cdot (y'_0 + y'_1) = z'_0 + z'_1$.

As before, the security of this derived PCG for multiplication triples also reduces to the Decisional QA-SD problem, since recovering x'_b and y'_b gives direct access to the original $\mathbf{x}_b$ used in the OLE-based PCG.

In [6], Bombar et al. showed how to convert multiplication triples over $\mathbb{F}_4$ into triples over $\mathbb{F}_2$. In [6, § 2.3], they presented a simple conversion technique using one bit of communication. From an $\mathbb{F}_2$ -triple and the associated bit, a party can reconstruct one of the inputs of the corresponding $\mathbb{F}_4$ triple and thus learn part of x_b from the underlying OLE correlation. In [6, § 2.4], they presented an improved protocol without communication and in this case an $\mathbb{F}_2$ -triple reveals both inputs of the corresponding $\mathbb{F}_4$ triple and all x_b from the underlying OLE correlation. In both cases, the security of the PCG for multiplication triples over $\mathbb{F}_2$ also breaks if one can solve Decisional QA-SD, with the same parameters (albeit with fewer evaluation points for [6, §2.3]).

PCGs for OLEs over $\mathbb{F}_2$ from [22]. In [22, § 2.3], Li et al. show how to convert OLEs over $\mathbb{F}_4$ into OLEs over $\mathbb{F}_2$ using the trace function $\text{Tr} : \mathbb{F}_4 \rightarrow \mathbb{F}_2$. From one OLE $x_0 \cdot x_1 = z_0 + z_1$ over $\mathbb{F}_4$, they show how to derive two OLEs over $\mathbb{F}_2$ as $\text{Tr}(x_0) \cdot \text{Tr}(x_1) = \text{Tr}(\alpha_0) + \text{Tr}(\alpha_1)$ and $\text{Tr}(\xi x_0) \cdot \text{Tr}(\xi x_1) = \text{Tr}(\beta_0) + \text{Tr}(\beta_1)$ (for appropriate $\alpha_0, \alpha_1, \beta_0$ and β_1 and a public $\xi \in \mathbb{F}_4$). From $\text{Tr}(x_0)$ and $\text{Tr}(\xi x_0)$, an adversary can readily reconstruct x_0 and if they can solve the Decisional QA-SD problem over $\mathbb{F}_4$, they can break the security of PCG for OLEs over $\mathbb{F}_2$ from [22]. This also applies to their more general technique to convert OLEs over $\mathbb{F}_{p^k}$ into OLEs over $\mathbb{F}_p$ (for any prime p and $k \geq 2$).

More advanced protocols. As mentioned in the introduction, PCGs have important applications in secure MPC. There exists a vast landscape of MPC protocol constructions and security models in which PCGs are employed. Our attacks can have catastrophic consequences for these protocols. As a simple example, consider a two-party BGW-style protocol [4] where the parties' inputs are first secret-shared using additive sharing, and then an arithmetic circuit is evaluated gate by gate using multiplication triples for each multiplication gate. A semi-honest player P_0 holding input x can break the privacy of an honest

player P_1 as follows: given two possible inputs y_0 and y_1 for P_1 , suppose P_0 runs honestly the protocol with P_1 and receives a share s of y_b . P_0 can then rerun the protocol with input x and that share and can reconstruct the two possible shares of P_1 as $y_0 - s$ and $y_1 - s$. P_0 is able to reconstruct two possible views of P_1 in the protocol (using these two shares) and reconstruct the two corresponding vectors of multiplicands from the multiplication triples used for each multiplication gate. He can then run our distinguisher against the Decisional QA-SD assumption to determine which of the two vectors could plausibly be an output of the PCG—thus guessing the bit b and violating the privacy of P_1 's input.

2.2 QA-SD as sparse multivariate polynomial interpolation

In the simple case where $c = 2$, adversaries against the search QA-SD problem are given (a_1, σ) with $\sigma := e_0 + a_1 e_1$, where a_1 is uniformly sampled in $\mathcal{R}$ and e_0, e_1 are random t -sparse elements of $\mathcal{R}$.

The starting point of all our attacks is the following simple observation. Take some $\mathbf{x} \in (\mathbb{F}_q^*)^n$; if $a_1(\mathbf{x}) = 0$, then $\sigma(\mathbf{x}) = e_0(\mathbf{x})$. The adversary knows the d^n evaluations of σ , and out of these a fraction about $1/q$ are such that $a_1(\mathbf{x}) = 0$. Because a_1 is public, the adversary easily obtains evaluations of the (secret) sparse polynomial e_0 on $\approx d^n/q$ randomly sampled points.

In the more general case where $c \geq 2$, then $\sigma = e_0 + a_1 e_1 + \dots + a_{c-1} e_{c-1}$, and $\sigma(\mathbf{x}) = e_0(\mathbf{x})$ whenever $a_1(\mathbf{x}) = \dots = a_{c-1}(\mathbf{x}) = 0$. The adversary then has access to $\approx d^n/q^{c-1}$ randomly sampled evaluations of e_0 . For instance, with $q = 4, c = 3, n = 14$ (the “aggressive” parameter sets of $\mathbb{F}_4\text{OLEAGE}$), the adversary knows $\approx 298,935$ evaluations of e_0 on randomly chosen points.

An algorithm that takes as input a large list of pairs $(\mathbf{x}_i, P(\mathbf{x}_i))$ and that behaves differently when $P \in \mathcal{R}$ is either very sparse or completely dense can therefore be turned into an adversary against the decisional QA-SD assumption. As we discuss more in detail in Section 3, such an algorithm does not seem to be readily available in the existing literature.

The search problem can be solved by interpolating the sparse polynomial e_0 given these many evaluations on random points. If e_0 is discovered, then it can be “peeled off” by writing $\sigma' := \sigma - e_0$. Then, when $a_1(\mathbf{x}) \neq 0$ and $a_2(\mathbf{x}) = \dots = a_{c-1}(\mathbf{x}) = 0$, we find that $\sigma'(\mathbf{x})/a_1(\mathbf{x}) = e_1(\mathbf{x})$. So the adversary has access to $\approx d^n(q-1)/q^{c-1}$ evaluations of e_1 (more than for e_0), and the process can continue until all the secret polynomials e_i have been recovered.

We now describe how to efficiently interpolate e_0 given a large number of evaluations on randomly chosen points. This leads to an efficient adversary against the search QA-SD assumption.

Sparse multivariate interpolation, one variable at a time. We can attack the QA-SD assumption by interpolating the t -sparse polynomial $e_0 \in \mathcal{R}_t$ given a large number of pairs $(\mathbf{x}_i, e_0(\mathbf{x}_i))_{1 \leq i \leq m}$, where all coordinates of the $\mathbf{x}_i$'s are non-zero. One possibility is to use an idea introduced by Zippel in the late 1970's [28]:

write e_0 as a (univariate) polynomial in X_1 ,

$$e_0 = \sum_{j=0}^{d-1} X_1^j P_j(X_2, \dots, X_n). \quad (4)$$

Write $\Omega = \{\mathbf{x}_i\}_{1 \leq i \leq m}$ the set of available evaluation points and let θ denote a generator of the multiplicative group of non-zero elements of $\mathbb{F}_q$. Then suppose that for some suffix $\mathbf{x}' \in (\mathbb{F}_q^*)^{n-1}$, the d vectors $(\theta^j | \mathbf{x}')$ all belong to Ω for $0 \leq j < d$. This allows to interpolate the univariate polynomial in equation (4) and obtain the values of $P_0(\mathbf{x}'), \dots, P_{d-1}(\mathbf{x}')$. This splits the initial t -sparse polynomial e_0 into d sparser “sub-polynomials” $P_0, \dots, P_{d-1}$. The t monomials of e_0 are dispatched (potentially irregularly) in the P_j ’s according to the power at which X_1 occurs in them.

There are d^{n-1} possible suffixes $\mathbf{x}'$ and for each of them, the probability that the d corresponding evaluation points are available is $q^{-d(c-1)}$. Therefore, we expect to obtain $d^{n-1}q^{-d(c-1)}$ evaluations of the P_j ’s. Taking again $q = 4, c = 3, n = 14$ ($\mathbb{F}_4$ OLEAGE/aggressive), the adversary easily obtains ≈ 389 evaluations of P_0, P_1 and P_2 on randomly chosen points.

Zippel also noted that it is easy to detect if any of the $P_j = 0$, because if it is the case, P_j always evaluates to zero. This happens when X_1^j never occurs in any of the t monomials of e_0 (X_1 may occur but with a different exponent). Based on the random choice of e_0 , it is easy to determine that $P_j = 0$ happens with probability $(1 - 1/d)^t$: there are d^t ways to choose the t exponents of X_1 in each of the t monomials, and only $(d - 1)^t$ avoid the exponent j .

With $q = 4$ (hence $d = 3$) and $t = 27$ ($\mathbb{F}_4$ OLEAGE/aggressive), $f_j = 0$ with probability 0.0000176.... Detecting if $P_j = 0$ is very efficient (nearly linear in the input size). This yields a practical recognizer for QA-SD samples. We note that this is already a practical attack that breaks roughly one instance over 56815 for the “aggressive” parameter set of $\mathbb{F}_4$ OLEAGE. Our aim in the rest of this article is to improve this distinguishing advantage.

Targetting all variables and all exponents. There are two ways in which this naive distinguisher can be improved. Firstly, instead of focusing on a single sub-polynomial P_j (i.e., the monomials in which X_1^j occurs), we could instead examine all the P_j ’s and detect if one of them is zero. Secondly, the univariate decomposition of eq. (4) could also be written using another variable X_i instead of X_1 . This yields the following theorem.

Theorem 1. *For constant q, c, t , the NAIVERECOGNIZER function shown in Figure 1 runs in expected time less than $d^n(n + c + q)$ (i.e., it is almost linear in the size of its input). The probability that it recognizes a QA-SD sample converges to 1 (exponentially fast) when n goes to infinity (for both the regular and the uniform noise distribution). As such, the decisional QA-SD assumption cannot hold for constant q, c and t .*

Proof. The running time of the function is easy to establish.

```

1: function NAIVERECOGNIZER( $a_1, \dots, a_{c-1}, \sigma$ )
2:   for  $1 \leq i \leq n$  do                                ▷ Write eq (4) with  $X_i$ 
3:     Set  $Zero_j \leftarrow 1$  for  $0 \leq j < d$               ▷  $Zero_j$  indicates if  $P_j = 0$ 
4:     for  $1 \leq j \leq d^{n-1}$  do                          ▷ enumerate all  $\mathbf{x}'$ 
5:        $b \leftarrow 1$                                     ▷ Can we evaluate the  $P_j$ 's on  $\mathbf{x}'$ ?
6:       for  $0 \leq k < d$  do                                ▷ Try the  $d$  points containing  $\mathbf{x}'$ 
7:          $t \leftarrow (n-1)$ -tuple over  $\{0, \dots, d-1\}$  of rank  $j$ 
8:          $t' \leftarrow (t[0:i], k, t[i:n-1])$ 
9:          $r_k \leftarrow$  rank of  $t'$  among  $n$ -tuples over  $\{0, \dots, d-1\}$ 
10:        for  $1 \leq \ell \leq c$  do                            ▷ Check all the public  $a_i$ 's
11:          if  $a_\ell[r_k] \neq 0$  then
12:             $b \leftarrow 0$ 
13:          else
14:             $y[k] \leftarrow \sigma[r_k]$                     ▷  $y[k] = e_0[r_k]$ 
15:          if  $b = 1$  then                                    ▷ interpolate eq (4) with  $X_i$ 
16:             $(u_0, \dots, u_{d-1}) \leftarrow \text{INTERPOLATE}([( \theta^0, y[0] ), \dots, ( \theta^{d-1}, y[d-1] )])$ 
17:            for  $0 \leq k < d$  do                            ▷  $u_k = P_k(\mathbf{x}')$ 
18:              if  $u_k \neq 0$  then
19:                 $Zero_k \leftarrow 0$ 
20:          for  $0 \leq j < d$  do
21:            if  $Zero_j = 1$  then
22:              return 1    ▷  $P_j = 0$ , and this should not happen for a dense  $e_0$ 
23:    return 0

```

Fig. 1. Simple recognizer for QA-SD samples.

There are $(d-k)^t$ ways to choose the exponents of X_i in the t monomials of e_0 that avoids k specific values. Therefore, using the inclusion-exclusion formula, we find that the probability, other the random choice of e_0 , that some X_i^j never occurs in e_0 , (i.e., that at least one of the P_j 's is zero) is

$$p = d^{-t} \sum_{k=1}^d (-1)^{k-1} \binom{d}{k} (d-k)^t$$

Clearly, $p \geq (1 - 1/d)^t$. This means that the probability, over the random choice of e_0 , that X_i^j never occurs in the t monomials of e_0 (for some $1 \leq i \leq n$ and $0 \leq j < d$) is $1 - (1 - p)^n$. This quantity tends to one when n increases, so that the (decisional) QA-SD assumption cannot hold for fixed (q, c, t) when n grows to infinity. $\square$

At this stage, we have to take into account the distribution of the sparse polynomials e_0 . For efficiency reasons, [7, 6] use a *regular* sparse distribution. Concretely, for $\mathbb{F}_4\text{OLEAGE}$ where $t = d^3$, this implies that e_0 can be written

$$\sum_{i=0}^{d-1} \sum_{j=0}^{d-1} \sum_{k=0}^{d-1} X_1^i X_2^j X_3^k Q_{ijk}(X_4, \dots, X_n),$$

where Q_{ijk} is a single monomial. It follows that writing e_0 as a univariate polynomial in either X_1, X_2 or X_3 as in eq. (4) leads to a balanced split where P_0, P_1 and P_2 all have exactly 9 monomials.

When e_0 is sampled using the regular sparse distribution, the first variables are therefore “tainted” and occur more regularly than the others. However, e_0 may split in an unbalanced way with the other remaining variables. The regular distribution of e_0 mean that we cannot hope to observe zero sub-polynomials P_j for the first $\log_d t$ variables, and the success probability of the above distinguisher drops to $1 - (1 - p)^{n - \log_d t}$.

This can be computed numerically for given values of t and d . For instance, again with $d = 3$ and $t = 27$ ($\mathbb{F}_4$ OLEAGE/aggressive) this yields $p = 0.000558\dots$ (i.e. it breaks one instance over 1792).

Distinguishing more monomials. In addition to the zero polynomial, it is easy to distinguish polynomials with one or two monomials. Indeed, a polynomial with a single monomial never vanishes (because all the input variables are non-zero). This makes it as easy to detect as the zero polynomial, because a random polynomial should vanish on a fraction $1/q$ of the inputs. Interpolating a single polynomial is also quite easy. Recall that θ is a generator of the multiplicative group $\mathbb{F}_q^*$. Suppose that $P = \theta^c X_1^{u_1} \dots X_n^{u_n}$, with $0 \leq c, u_i < d$. Given a (non-zero) input-output pair $\mathbf{x} = (\theta^{v_1}, \dots, \theta^{v_n})$ and $y = \theta^b$, we obtain a linear equation in the exponents: $b = c + u_1 v_1 + \dots u_n v_n$. Therefore at least $n + 1$ evaluations points are required for the interpolation, and a few extra points can be used to confirm the result. The interpolation itself amounts to solving a linear system in $(n + 1)$ variables.

Next, let P denote a polynomial with two monomials: $P = \alpha_1 + \alpha_2$. The main observation is that P can be factored as $P = \alpha_1(1 + \alpha_2/\alpha_1)$. Note that α_2/α_1 is a monomial in $\mathcal{R}$, because the exponents are defined “modulo d ”. When $P(\mathbf{x}) = 0$, then necessarily $(\alpha_2/\alpha_1)(\mathbf{x}) = -1$ because $\alpha_1(\mathbf{x}) \neq 0$. This provides evaluations of the monomial α_2/α_1 , which can then be interpolated. However, there are always going to be several solutions, namely α_1/α_2 , α_2/α_1 and the constant monomial equal to -1. Therefore the technique described above (interpolating a single monomial by solving a linear system in the exponents) will produce a linear system with a non-trivial kernel. In our experiments, it was always of small dimension (1 or 2) and could be searched exhaustively. Given a candidate for α_2/α_1 , we may compute $\alpha_1(\mathbf{x}) = P(\mathbf{x})/(1 + \alpha_2(\mathbf{x})/\alpha_1(\mathbf{x}))$ for all points where $P(\mathbf{x}) \neq 0$. This makes it possible to interpolate α_1 , or to move on to the next candidate if this process fails. Finally, α_2 can be reconstructed given of α_2/α_1 . This technique requires $\approx q(n + 1)$ evaluations of P .

To summarize, we are capable of efficiently detecting polynomials with 0, 1 or 2 monomials. This yields an efficient recognizer for QA-SD samples. What is the probability that eq. (4) leads to a decomposition where some P_j has less than two monomials? Write r_j the number of monomials of e_0 that contain X_i^j ; we ask what is the probability that $\min r_j \leq 2$. The random choice of e_0 implies that $(r_0, \dots, r_{d-1})$ follows an equiprobable multinomial distribution of parameters $(t; 1/d, \dots, 1/d)$. Our problem amounts to finding the distribution of the minimum of a multinomial vector. This corresponds to the situation where t balls are thrown uniformly at random into d bins: the number of balls in each bin (the “occupancy numbers”) have precisely this multinomial distribution.

While the maximum load has been widely studied because of its implication on the performance of hash tables with closed addressing, the minimum load has received much less attention. Some theoretical results are readily available [14, 8], but in our case we can simply estimate the distribution of the minimum load using Monte-Carlo sampling.

Denote by $p(d, n, t, 1, t')$ the probability (over the random choice of e_0) that the minimum of the number of terms of the sub-polynomial $P_0, \dots, P_{d-1}$ that appear in eq. (4) is equal to t' , taken over all possible variables X_i . Denote by $p'(d, n, t, 1, t')$ the same quantity under the regular noise distribution. Using one million random trials, we determined the distribution of $p'(3, 14, 27, 1, t')$, shown in Table 2. The probability that this simple recognizer for 0, 1 or 2 monomials succeeds when trying P_0, P_1 and P_2 for all the 14 variables (included the first 3 that are tainted) is 5.8%. This breaks one instance over 17 in practice.

Looking at other examples, [22] proposed other concrete parameters over $\mathbb{F}_4$. More specifically, the parameter set $q = 4, n = 19, c = 5, t = 17$ (with uniform noise distribution) is claimed to offer 80 bits of security. It is slightly different from the parameters of $\mathbb{F}_4\text{OLEAGE}$: c is larger and t is smaller, even though the ct product is comparable. This has a non-trivial effect on security. We obtain 23 evaluation points for each of the 3 subpolynomials P_0, P_1 and P_2 in eq. (4) on expectation. This is sufficient to determine if one the sub-polynomial P_j is either always zero or never zero. We again empirically determined the probability $p(3, 19, 17, 1, t')$ that the smallest sub-polynomial has t' terms, also shown in Table 2. Sub-polynomials with zero or one monomials can be detected (there are not enough evaluation points to detect polynomials with two monomials using the technique described above). Therefore, this simple recognizer will detect instances of QA-SD with probability 42.82%, and the parameters are practically insecure.

[22] also proposed concrete parameters over larger fields, specifically $\mathbb{F}_8$, namely $q = 8, n = 10, c = 2, t = 49$ (with regular noise distribution), supposed to offer 80 bits of security. Over the 7^{10} total evaluation points, a_1 vanishes on a fraction $1/8$, so the sparse polynomial e_0 can be evaluated on $\approx 35\text{M}$ points. Writing

t'	$p'(3, 14, 27, 1, t')$	$p(3, 19, 17, 1, t')$	$p'(7, 10, 49, 1, t')$
0	0.06%	5.62%	2.90%
1	0.78%	37.20%	20.89%
2	4.95%	50.40%	48.65%
3	18.24%	6.77%	26.19%
4	37.23%	0.01%	1.37%
5	31.70%	0.00%	0.00%
6	6.89%	0.00%	0.00%
7	0.15%	0.00%	0.00%
8	0.00%	0.00%	0.00%

Table 2. Probability that e_0 splits with a sub-polynomial of t' terms when writing eq. (4) for each possible variable X_i , for the regular noise distribution.

the decomposition of eq. (4) allows us to evaluate the 7 sub-polynomials on 134 points. This is more than enough to detect if they have 0, 1 or 2 monomials. We again determined empirically the probability that this succeeds, reported in table 2. This yields an efficient recognizer with success probability 72.4%. These parameters too are practically insecure.

Interpolating more variables simultaneously. Instead of writing e_0 as a univariate polynomial as in Eq. (4), we could write it as a bivariate polynomial in the first two variables it as follows:

$$e_0 = \sum_{i=0}^{d-1} \sum_{j=0}^{d-1} X_1^i X_2^j P_{ij}(X_3, \dots, X_n). \quad (5)$$

This splits e_0 into d^2 sub-polynomials in $n-2$ variables. If e_0 could be evaluated on the d^2 vectors $(\theta^i, \theta^j | \mathbf{x}')$, then the P_{ij} 's could be evaluated on $\mathbf{x}'$. Because e_0 can only be evaluated on $\mathbf{x}$ when $a_1(\mathbf{x}) = \dots = a_{c-1}(\mathbf{x}) = 0$, each evaluation point $\mathbf{x}$ is available with probability $q^{-(c-1)}$. This yields $d^{n-2} q^{-(c-1)d^2}$ evaluation points for the d^2 subpolynomials, on expectation (over the random choice of the a_i 's). This generalizes to writing e_0 as a polynomial in $n' < n$ variables, whose coefficients are polynomials in the remaining $n - n'$ variables, yielding $d^{n-n'} q^{-(c-1)d^{n'}}$ evaluation points for each one of the $d^{n'}$ sub-polynomials that appears as coefficients of $X_1, \dots, X_{n'}$. Of course, this can be also be done for any over n' -subset of the variables. The interest of this maneuver is that it splits e_0 into smaller pieces that have a greater chance to be unbalanced.

A simple distinguishing attack can therefore be written as follows. For each subset of n' variables among $X_1, \dots, X_n$: obtain m evaluation points for all the P_j 's then for each P_j : try to detect if it has 0, 1 or 2 monomials; in case of success return 1; after all the combinations have been tried, return 0. This algorithm is efficient, and we now discuss its success probability. Denote again by $p(d, n, t, n', t')$ the probability that the smallest sub-polynomial obtained by decomposing e_0 as a n' -variable polynomial has t' terms, considering all the $\binom{n}{n'}$ n' -subsets of the n variables. Write $p'(d, n, t, n', t')$ the same probability under the regular noise distribution. We empirically determined these probabilities (using Monte-Carlo sampling) in cases of interest.

For instance, let us consider several parameter sets proposed in [22], with $q = 4$, and summarized in Table 3. They are supposed to offer 80 or 128 bits of security. The practical attack described above yields distinguishers with advantage larger than 40% in all cases.

Going further: an algorithm for non-trivial interpolation problems.

In the next section, we describe an algorithm capable of interpolating sparse polynomials with more than two monomials. This significantly improves the success probability of the distinguishers presented up to this point and the range of parameters that can be attacked. For instance, being able to reconstruct polynomials with at most 4 monomials over 14 variables over $\mathbb{F}_4$ enables us to break instances of $\mathbb{F}_4\text{OLEAGE/aggressive}$ with probability 61% (looking at Table 2).

3 Sparse polynomial interpolation with known input/output pairs

Let $P \in \mathbb{K}[X_1, \dots, X_n]$ be a t -sparse polynomial over a field $\mathbb{K}$. The sparse interpolation problem asks, given a set of pairs $((\mathbf{x}, P(\mathbf{x})) \in \mathbb{K}^n \times \mathbb{K}$, to recover P . How these pairs are obtained depends on the model considered to solve the problem. Since the 1980s, the most studied model provides access to an oracle that allows evaluations of P at any chosen point. We call it the black-box model. In this framework, efficient algorithms for the task of sparse interpolation have been designed [28, 5, 21, 18, 17, 3, 19, 26]. The model considered in this paper requires that each available pair $(\mathbf{x}, P(\mathbf{x}))$ be sampled uniformly at random. We call it the random points model. It is important to note that algorithms for sparse interpolation in the black-box model do not extend to the random points model, since in the latter one cannot choose the evaluation points arbitrarily. Under the random points model with $\mathbb{K} = \mathbb{C}$, Candès et al. [11] show that sparse interpolation reduces to an ℓ_1 -norm minimization problem in an underdetermined linear system. More details on this are given in subsection 3.2. In this section, we propose a method for solving the sparse interpolation problem in the random points model over a finite field $\mathbb{K} = \mathbb{F}_q$. Moreover, our problem includes the constraint that in each available random pair $(\mathbf{x}, P(\mathbf{x}))$ the vector $\mathbf{x}$ has no zero component. More formally:

Problem 1. Let $P \in \mathbb{F}_q[X_1, \dots, X_n]$ be a unknown polynomial such that $\forall i \in \{1, \dots, n\}$, $\deg_{X_i}(P) \leq (q-2)$, $wt(P) = t$. Given $m > 0$ pairs $(\mathbf{x}_i, P(\mathbf{x}_i))$ where the $\mathbf{x}_i$'s are independently and uniformly sampled from $(\mathbb{F}_q^*)^n$ the goal is to reconstruct P (i.e. recover its coefficients).

Sudan [25] has developed an algorithm to solve a closely related problem:

Problem 2 ([25]). Let $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)$ be data points in $\mathbb{K}^n \times \mathbb{K}$. Assume the inputs $\mathbf{x}_1, \dots, \mathbf{x}_N$ are all distinct and together span H^n where H is an arbitrary finite subset of $\mathbb{K}$. The problem is to compute all polynomials $P \in \mathbb{K}[X_1, \dots, X_n]$ of total degree at most D that agree with m of these data points.

c	t	n	Error type	λ	n'	$E[m]$	p_0	p_1	p_2
2	42	16	non-regular	80	2	18	99.7%	0.03%	0% (*)
2	81	19	regular	128	2	13301	7.24%	46.98%	43.58%
2	66	17	non-regular	128	2	54	37.82%	59.56%	2.62% (*)

Table 3. Parameter sets from [22]. λ denotes the expected security level. n' denotes the number of interpolated variables. $E[m]$ denotes the expected number of evaluation points for the sub-polynomials P_j 's. p_0, p_1 and p_2 denote $p(3, n, t, n', 0)$, $p(3, n, t, n', 1)$ and $p(3, n, t, n', 2)$ respectively, or the variant with regular error distribution when applicable. The (*) means that there are not enough evaluation points to detect two monomials.

By solving Problem 2 with $D = n(q - 2)$ and $H = \mathbb{F}_q \setminus \{0\}$, one can solve 1. Sudan's algorithm solves Problem 2 in time $\text{poly}((q - 1)^n)$ provided that:

$$m \geq (n + 1)(D + 1)^{\frac{1}{n+1}} (q - 1)^{(n - \frac{1}{n+1})}$$

However, the lower bound on the required number of evaluation points is excessively large, rendering the method impractical for the parameters from [6, 22, 7].

In Problem 1, if we drop the constraint that each available random pair $(\mathbf{x}_i, P(\mathbf{x}_i))$ has no zero component in $\mathbf{x}_j$, the problem becomes a special case of the following one described by Wasserman [27]:

Problem 3 ([27]). Let $P \in \mathbb{K}[X_1, \dots, X_n]$ be a unknown polynomial of total degree $\leq D$ over a finite field $\mathbb{K}$. We are given data-points $(\mathbf{x}_1, \mathbf{y}_1), \dots, (\mathbf{x}_N, \mathbf{y}_N) \in \mathbb{K}^n \times \mathbb{K}$. An unknown size- m subset $(\mathbf{x}_{i_1}, y_{i_1}), \dots, (\mathbf{x}_{i_m}, y_{i_m})$ of the data-points satisfy $y_{i_k} = P(\mathbf{x}_{i_k})$ for uniformly and independently picked $\mathbf{x}_{i_k} \in \mathbb{K}^n$. The goal is to reconstruct from this data a small set of polynomials including P .

More precisely, solving Problem 3 with $N = (q - 1)^n$ and $D = n(q - 2)$ allows one to solve Problem 1. To solve Problem 3, Wasserman proposes an algorithm [27] that returns $\text{poly}(m)$ candidate polynomials in time $\text{poly}(m)$. However, this algorithm only works if

$$|\mathbb{K}| \geq 3\mathcal{L} \text{ and } m \geq \frac{2}{n!} \left(\frac{n+1}{2} + \mathcal{L} \right)^n$$

with $\mathcal{L} = ((n + 1)ND)^{\frac{1}{n+1}}$. In the parameters proposed in [6, 22, 7], the field sizes are always less than 16, and these two conditions are not satisfied. To the best of our knowledge, aside from these two algorithms, none exists for solving Problem 1 in a non-trivial way.

In this section, we propose a method for solving Problem 1 under the conditions $q > 2$ and $2 < t < n$, with the following high-level structure:

1. Embed the problem 1 in $\mathbb{C}$.
2. Solve it in $\mathbb{C}$.
3. Recover the solution over $\mathbb{F}_q$ from the solution over $\mathbb{C}$.

3.1 Complex lifting

We consider the ring of n -variate polynomials $\mathbb{F}_q[X_1, \dots, X_n]$ over $\mathbb{F}_q$ and $P \in \mathbb{F}_q[X_1, \dots, X_n]$ a polynomial with $t \geq 1$ monomials. We denote

$$P(X_1, \dots, X_n) = P(\mathbf{X}) = \sum_{j=1}^t u_j X_1^{\alpha_{j,1}} \dots X_n^{\alpha_{j,n}} = \sum_{j=1}^t u_j \mathbf{X}^{\alpha_j} \quad (6)$$

where $\alpha_j = (\alpha_{j,0}, \dots, \alpha_{j,n-1})$ is a vector of integers in $\{0, \dots, d-1\}$ and $u_j \in \mathbb{F}_q^*$ for all $j \in \{1, \dots, t\}$. The following theorem asserts the existence of an explicit

n -variate polynomial $F \in \mathbb{C}[Z_1, \dots, Z_n]$ over the complex numbers with d^{t-1} monomials that “interpolates” the values of P by *lifting* the evaluation points and values to $\mathbb{C}$. This is done using a group homomorphism $\chi : \mathbb{F}_q^* \rightarrow \mathbb{C}^*$ that maps a generator of $\mathbb{F}_q^*$ to a primitive $(q-1)$ -root of unity in $\mathbb{C}$, extended to $\mathbb{F}_q$ with $\chi(0) = 0$. Such a map is called a *primitive multiplicative character* of $\mathbb{F}_q$.

Theorem 2. *Let q be a prime power and $n, t \geq 1$ be two integers. Let χ be a primitive multiplicative character of $\mathbb{F}_q$. Let $P \in \mathbb{F}_q[X_1, \dots, X_n]$ a polynomial with $t \geq 1$ monomials. There exists a polynomial $F \in \mathbb{C}[Z_1, \dots, Z_n]$ with at most d^{t-1} monomials such that for all $(\gamma_1, \dots, \gamma_n) \in (\mathbb{F}_q^*)^n$,*

$$F(\chi(\gamma_1), \dots, \chi(\gamma_n)) = \chi(P(\gamma_1, \dots, \gamma_n)).$$

Moreover, if P is defined as in (16), we have

$$F(\mathbf{Z}) = F(Z_1, \dots, Z_n) = \sum_{\mathbf{i} \in \{0, \dots, d-1\}^t} (c_{\mathbf{i}} \chi(u_1^{i_1} \dots u_t^{i_t})) \cdot \mathbf{Z}^{i_1 \alpha_1 + \dots + i_t \alpha_t \bmod d}$$

where the coefficients $c_{\mathbf{i}}$ satisfy the following conditions:

1. $c_{\mathbf{i}} = 0$ if $i_1 + \dots + i_t \not\equiv 1 \pmod{d}$;
2. $c_{\mathbf{i}} = (-1/d)^{t-1}$ if $i_l = 1$ for some $l \in \{1, \dots, t\}$, and $i_k = 0$ for $k \in \{1, \dots, t\} \setminus \{l\}$;
3. $|c_{\mathbf{i}}| > 1/d^{t-1}$ otherwise.

The proof of this theorem relies on simple computation and well-known facts on Gauss and Jacobi sums (see e.g., [20, Chapter 8] or [23, Chapter 5]). It is provided as supplementary material

3.2 Sparse complex interpolation by “Compressed sensing”

We have now reduced our initial problem to the one of reconstructing a sparse polynomial $F \in \mathbb{C}[Z_1, \dots, Z_n]$. To achieve this, we use results from signal theory.

Around 2005, Candès et al. [11] introduced the idea that if a discrete signal $\mathbf{g} = (g(0), \dots, g(N-1)) \in \mathbb{C}^N$ is sparse enough, then it can be entirely reconstructed with fewer measurement (or evaluation) than expected. This idea was later developed (see e.g., [13, 16, 12]) into what is known as the *Compressed Sensing* theory, a powerful tool in signal recovery.

In this section, we show that early results from Candès et al. [11] can be used directly to reconstruct our sparse polynomial $F \in \mathbb{C}[Z_1, \dots, Z_n]$, with only a few evaluation points. More precisely, we use the following result, which is a special case of [11, Theorem 1.3].

Theorem 3. *With high probability depending on the choice of a constant γ , any polynomial $F \in \mathbb{C}[Z_1, \dots, Z_n]$ with at most d^{t-1} monomials can be exactly reconstructed with only m uniformly random evaluation points, provided that $m \geq \gamma d^{t-1} n \log(d)$.*

To understand where this come from, we first need to recall some fundamentals regarding signal theory. Let $\mathbf{g} = (g(0), g(1), \dots, g(N-1)) \in \mathbb{C}^N$ be the vector representation of discrete-time signal whose coefficients are given by a (truncated) sequence g over $\mathbb{C}$. The goal is to reconstruct the vector $\mathbf{g}$ from a set of m measurements y_k where

$$y_k = \hat{g}(k) = \sum_{s=0}^{N-1} g(s) e^{-2i\pi s/N}.$$

is the DFT of g at point k . Setting $\mathbf{a}_k = (1, e^{-2i\pi k/N}, \dots, e^{-2i\pi k(N-1)/N})$ the above equation can be re-written as:

$$y_k = \langle \mathbf{g}, \mathbf{a}_k \rangle. \quad (7)$$

Assume that we have access to m of such measurements. It is clear that if $m = N$ $\mathbf{g}$ can be reconstructed by simple linear algebra. But what happen when we are given access to less than N measurements? As it was shown in [11], if $\mathbf{g}$ is sparse enough, then it can be reconstructed with fewer measurements than one would expect. More precisely

Lemma 1 (From [11], Theorem 1.3). *If Ω is a uniformly random subset of $\{0, \dots, N-1\}$ of cardinality m and if $\mathbf{f}$ is an S -sparse signal in $\mathbb{C}^N$, then $\mathbf{f}$ is the solution of the following convex minimisation problem:*

$$\begin{aligned} & \underset{\mathbf{z} \in \mathbb{C}^N}{\text{minimize}} && \|\mathbf{z}\|_1 \\ & \text{subject to} && \mathbf{y}_k = \langle \mathbf{z}, \mathbf{a}_k \rangle, \forall k \in \Omega, \end{aligned} \quad (8)$$

with high probability provided that $m \geq \gamma S \log(N)$ for some constant γ .

The constant γ influences the accuracy of the model. The authors of [11] estimate that for a given constant γ , the probability that there is a unique solution to Problem 8, is at least $1 - \mathcal{O}(N^{-\gamma/23+1})$. However, as they state, this estimation is not optimal. In practice, they mention that more than 50 % of the signals can be recovered by taking $\gamma = 4$, and more than 90 % when $\gamma = 8$.

Now, how do all of this relate to our initial problem? Recall that we are given as input a polynomial $F \in \mathbb{C}[Z_1, \dots, Z_n]$ of degree at most d in each variable. The coefficients of this polynomial can be modeled as a discrete signal $\mathbf{g} \in \mathbb{C}^{d^n}$. Furthermore, from Theorem 2, we know that F has at most d^{t-1} monomials, which makes our signal d^{t-1} -sparse. Theorem 3 comes naturally from here.

To sum things up, we have now reduced the problem of finding the monomials of F to the ℓ_1 -norm minimisation problem of the solution of a linear system with N variables and m equations, $A\mathbf{z} = \mathbf{y}$. This allows us to reformulate the convex minimisation problem given in Lemma 1 as follows:

$$\begin{aligned} & \underset{\mathbf{z} \in \mathbb{C}^N}{\text{minimize}} && \|\mathbf{z}\|_1 \\ & \text{subject to} && A\mathbf{z} = \mathbf{y}, \end{aligned} \quad (9)$$

with $N = d^n$. Note that this problem is defined over $\mathbb{C}^N$. Next, we show how it can be reduced to a convex minimization problem over $\mathbb{R}$.

3.3 Convex minimisation problems over $\mathbb{R}$

Let $\mathbf{z} = \mathbf{u} + \mathbf{i}\mathbf{v}$ be a vector of $\mathbb{C}^n$. For all $A \in \mathbb{C}^{m \times N}$, we denote by $\Re A$ and $\Im A$ the matrices containing, respectively, the real and imaginary parts of the entries of A . We will now define the following quantities:

$$\tilde{\mathbf{z}} = \begin{pmatrix} \mathbf{u} \\ \mathbf{v} \end{pmatrix} \in \mathbb{R}^{2N}, \quad \tilde{A} = \begin{pmatrix} \Re A & -\Im A \\ \Im A & \Re A \end{pmatrix} \quad \text{and} \quad \tilde{\mathbf{y}} = \begin{pmatrix} \Re \mathbf{y} \\ \Im \mathbf{y} \end{pmatrix}$$

From the definition of the ℓ_1 -norm given in eq. 3, Problem (9) is equivalent to

$$\begin{aligned} & \underset{\mathbf{z} \in \mathbb{R}^{2N}}{\text{minimize}} && \sum_{j=1}^N \sqrt{\tilde{z}_j^2 + \tilde{z}_{j+N}^2} \\ & \text{subject to} && \tilde{A}\tilde{\mathbf{z}} = \tilde{\mathbf{y}} \end{aligned} \tag{10}$$

which is a convex minimization problem in $\mathbb{R}$.

Note that not all convex optimization problems can be solved in polynomial time (e.g., [15]). However, problem (10) is a particular case known as *Second-Order Cone Programming* (SOCP). To properly define what this SOCP instance, we first need to explain what a second-order cone is. We restrict ourselves to the case of the *three-dimensional second-order cone*, as this is the one we need, and define it as the set

$$\mathcal{K} = \left\{ (a, b, c) \in \mathbb{R}^3 \mid \sqrt{b^2 + c^2} \leq a \right\}.$$

Now, let $\mathbf{r} = (r_1, \dots, r_N) \in \mathbb{R}^N$ be a vector of auxiliary variables. Consider the following constraints:

$$\forall j \in \{1, \dots, N\}, \quad \sqrt{\tilde{z}_j^2 + \tilde{z}_{j+N}^2} \leq r_j.$$

Since for all $j \in \{1, \dots, N\}$, $(r_j, \tilde{z}_j, \tilde{z}_{j+N})$ belongs to the cone $\mathcal{K}$, we say that each constraint is conic. We can now define the following problem:

$$\begin{aligned} & \underset{\mathbf{r} \in \mathbb{R}^N}{\text{minimize}} && \sum_{j=1}^N r_j \\ & \text{subject to} && \tilde{A}\tilde{\mathbf{z}} = \tilde{\mathbf{y}}, \\ & && (r_j, \tilde{z}_j, \tilde{z}_{j+N}) \in \mathcal{K}, \quad j = 1, \dots, N. \end{aligned} \tag{11}$$

The problem (11) is a SOCP and it has the same optimal solution as (10). For any feasible $\mathbf{z}$ for (10), setting $r_j = \sqrt{\tilde{z}_j^2 + \tilde{z}_{j+N}^2}$ yields a feasible $(\mathbf{z}, \mathbf{r})$ for (11) with the same objective, so the optimal value of (11) cannot exceed that of (10). Conversely, an optimal $(\mathbf{z}^*, \mathbf{r}^*)$ of (11) yields a feasible $\mathbf{z}^*$ for (10) so the optimal value of (10) cannot exceed that of (11). Hence both problems attain exactly the same optimal value and are equivalent.

As shown by Nesterov and Nemirovski [24], interior-point methods can solve second-order cone programs in polynomial time in N . To solve our SOCP, we employ the homogeneous and self-dual algorithm [1] implemented in the MOSEK [2] software library—a specific interior-point method tailored for conic optimization problems such as SOCPs. While MOSEK theoretically solves the optimization problem in time $\mathcal{O}(N^{3.5})$, in practice it runs far faster by employing a variety of techniques to accelerate its performance.

All in all, we have shown that solving the SOCP (11) gives a solution the convex problem (10) over $\mathbb{R}$, which in turns provides a solution to the ℓ_1 -minimization problem (9) over $\mathbb{C}$. As such if the solver converges to the optimal solution for Problem 11, we are able to recover the sparse polynomial F described in subsection 3.1 (as explained in subsection 3.2).

3.4 Reconstruction of the solution in $\mathbb{F}_q[X_1, \dots, X_n]$

Let $P : \mathbb{F}_q^n \rightarrow \mathbb{F}_q$ be a t -sparse polynomial over $\mathbb{F}_q$. Let θ denote a generator of the multiplicative group of non-zero elements in $\mathbb{F}_q$. Set $\omega := e^{\frac{2\pi i}{d}}$. Let χ denote the multiplicative character of $\mathbb{F}_q$ that maps θ^r to ω^r , extended with $\chi(0) = 0$.

We assume that we are given m pairs $(\mathbf{x}_j, y_j)$ where each $\mathbf{x}_j$ is a random n -vector with coordinates in $\{0, \dots, d-1\}$ and $y_j = P(\theta^{x_{j1}}, \dots, \theta^{x_{jn}})$. Our complete interpolation algorithm is shown in Fig. 2.

It begins by “lifting the problem to $\mathbb{C}$ ” and finding a (sparse) complex polynomial F in n variable with $\deg_{X_j}(F) \leq d-1$ that correctly interpolate the complex lifting of the input pairs (as illustrated by Fig. 3). More precisely, it is such that $F(\omega^{x_{j1}}, \dots, \omega^{x_{jn}}) = \chi(y_j)$. The coefficients of F are stored in $\mathbf{z}$ after step I3. They are obtained by ℓ_1 -norm minimization, and therefore we expect F to be the sparsest complex polynomial correctly interpolating the inputs. In fact, we expect F to be the polynomial promised by Theorem 2.

To reconstruct the t coefficients of P from the d^n coefficients of F , we perform an exhaustive search on t and we assume that F satisfies the conclusions of Theorem 2. Under these assumptions, the t monomials of P corresponds to t coefficients of F of norm d^{1-t} (by point 2 of Theorem 2). Moreover, other coefficients of F are either zero or have norm larger than d^{1-t} (by point 3 of Theorem 2). Because the coefficients of F are computed numerically using floating point numbers with finite precision, we use a kind of “fuzzy matching” to recover the monomials of P by choosing those whose coefficients have norm closest to d^{1-t} .

Once the support of P has been determined, finding the coefficients of the t monomials is a simple matter. And once a full t -sparse polynomial has been assembled, it is easy to test if it is correct by checking that it correctly interpolates the inputs over $\mathbb{F}_q$. The running time of this process is dominated by the invocation of the SOCP solver, whose running time is essentially $\mathcal{O}(m^2 d^n)$, where m is the number of available random pairs $(\mathbf{x}_j, P(\mathbf{x}_j))$. Its space complexity is essentially that of storing A and z : this makes $16(m+1)d^n$ bytes using double-precision complex numbers.

Algorithm I (*Sparse interpolation in the random points setting*). Returns a sparse representation of f .

- I1.** [Setup.] Allocate a $m \times d^n$ matrix A over $\mathbb{C}$ (in practice: double-precision complex float-point numbers) as well as a m -vector b over $\mathbb{C}$. Set $t \leftarrow 1$.
- I1.** [Assemble.] Enumerate all n -tuples with values in $\{0, \dots, d-1\}$; denote by α_j the j -th in lexicographic order; then for $1 \leq i \leq m$: set $A[i, j] \leftarrow \omega^{\langle \alpha_j, \mathbf{x}_i \rangle}$ and $b_i \leftarrow 0$ if $y_i = 0$ or $b_i \leftarrow \omega^{y_i}$ otherwise.
- I3.** [Minimization.] Run a SOCP solver to find $\mathbf{z} \in \mathbb{C}^{d^n}$ such that $A\mathbf{z} = \mathbf{b}$ and $\|\mathbf{z}\|_1$ is minimal.
- I4.** [Guess support.] Evaluate $||z_i| - d^{1-t}|$ on all indices $1 \leq i \leq d^n$ and let $j_1, \dots, j_t$ denote the t indices that yield the smallest values. Write $\alpha_1, \dots, \alpha_t$ the n -vectors with coordinates in $\{0, \dots, d-1\}$ of respective ranks $j_1, \dots, j_t$ [this is a guess of the monomials occurring in P].
- I5.** [Interpolate.] Assemble the $m \times t$ matrix C over $\mathbb{F}_q$ such that $C[i, j] = \theta^{\langle \mathbf{x}_i, \alpha_j \rangle}$. Solve the linear system $C\mathbf{u} = \mathbf{y}$ over $\mathbb{F}_q$. If no solution exists, jump to step I7.
- I6.** [Validate.] Check if the t -sparse polynomial $\sum_i u_i X_1^{\alpha_{i1}} \dots X_n^{\alpha_{in}}$ evaluates to y_i on input x_i , for all $1 \leq i \leq m$. If so, return it and stop the algorithm.
- I7.** [Failure.] Increment t . If $t = n$, return $\perp$ and stop the algorithm. Otherwise, return to step I4.

Fig. 2. Sparse multivariate interpolation algorithm over $\mathbb{F}_q$ in the random points setting.

3.5 Towards a reduced memory complexity

SOCP solvers rely on interior points method and only read the matrix A . In our case, A has a compact representation: each $A[i, j] = \omega^{\langle \mathbf{x}_i, \mathbf{z}_j \rangle}$ can be computed given the mn coefficients of the input evaluation points $\mathbf{x}_i$'s (the $\mathbf{z}_j$'s range across all possible monomials). This opens the possibility of designing an ad hoc SOCP solver that does not require the explicit representation of A , potentially dividing the memory complexity by k .

This possibility is made credible by the fact that A intervenes in interior points method in the “direction search” step, where a $(m + d^n) \times (m + d^n)$ linear system of the following shape has to be solved:

$$\begin{pmatrix} H & A^t \\ A & 0 \end{pmatrix} \begin{pmatrix} \mathbf{v} \\ \mathbf{w} \end{pmatrix} = \begin{pmatrix} \mathbf{g} \\ \mathbf{h} \end{pmatrix}. \quad (12)$$

Here H is symmetric positive semi-definite and sparse (block-diagonal).

The following derivation is given in [9, §10.4.2] and in [1]. In a solution of (12), we have $H\mathbf{v} + A^t\mathbf{w} = \mathbf{g}$ and $A\mathbf{v} = \mathbf{h}$. Thus we have $\mathbf{v} = H^{-1}(\mathbf{g} - A^t\mathbf{w})$ so that substituting in the second equation yields $AH^{-1}(\mathbf{g} - A^t\mathbf{w}) = \mathbf{h}$, from which we derive $\mathbf{w} = (AH^{-1}A^t)^{-1}(AH^{-1}\mathbf{g} - \mathbf{h})$. So the system can be solved as follows:

1. Compute a sparse Choleski factorization $H = LL^t$.

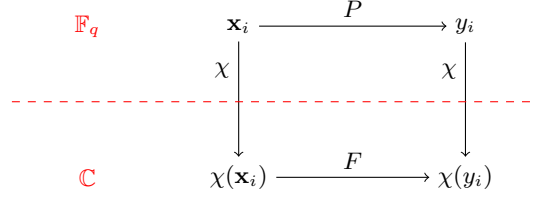


Fig. 3. Illustration of our algorithmic strategy.

2. Form Schur complement $S \leftarrow AH^{-1}A^t$. Note that this is an $m \times m$ matrix. For $1 \leq i \leq n$, solve $LL^t \mathbf{u} = A_i$ and compute $A\mathbf{u}$ (this yields the i -th column of S).
3. Compute a Choleski factorization $S = MM^t$.
4. Solve $LL^t \mathbf{u} = \mathbf{g}$, then solve $MM^t \mathbf{w} = A\mathbf{u} - \mathbf{h}$.
5. Solve $LL^t \mathbf{v} = \mathbf{g} - A^t \mathbf{w}$.

In this algorithm, the space requirement is reduced to m^2 (for S), plus $\mathcal{O}(d^n)$ (to hold a few dense vectors), plus the space required to hold the Choleski factorization of H (which is expected to be modest). It follows that (12) can be solved efficiently, and with memory proportional to the size of the solution vector when A admits a compact representation (recomputing A_{ij} “on the fly” when needed). We leave an efficient implementation of Algorithm I with reduced memory as a perspective for future work.

3.6 Experimental results

Let P be the sparse polynomial in $\mathbb{F}_q[X_1, \dots, X_n]$ that we aim to recover and m the number of random pairs $(\mathbf{x}_j, P(\mathbf{x}_j)) \in \mathbb{F}_q^n \times \mathbb{F}_q$ at our disposal. An important question is “what is the minimal value m_0 of m that guarantees to recover P ?” We already provided a theoretical answer to this question in Section 3.2, but it is not precise enough. However, we can give an experimental estimation of this value.

Since recovering P involves solving an SOCP, we used the MOSEK solver. Given q, n, t and m , let $R(q, n, t, m)$ denote the event that P be recovered. To estimate $\Pr(R(q, n, t, m))$ we need to rerun the solver K times and define N_R as the number of runs for which it converges to the optimal solution. We then define $\Pr(R(q, n, t, m)) = \frac{N_R}{K}$. Because the solver has high memory requirements, we used $K = 1000$ for the smaller values of n and $K \leq 100$ for the larger ones. For given parameters q, n and t , let

$$m_{90\%}(q, n, t) = \min\{m \mid \Pr(R(q, n, t, k)) \geq 0.90, \forall k \in \{m, \dots, m + 10\}\}$$

To determine the values of $m_{90\%}(q, n, t)$, for $q = 4, t = 3$ and $n \in \{6, 7, 8, 9, 10\}$, we applied the following algorithm by choosing $K = 1000$ for $n \in \{6, 7, 8\}$ and $K = 100$ for $n \in \{9, 10\}$. Let L be the cumulative distribution function of

Algorithm II ($m_{90\%}$ estimator). Returns $m_{90\%}(q, n, t)$ and $[(1, \Pr(R(q, n, t, 1))), \dots, (m_{90\%} + 10, \Pr(R(q, n, t, m_{90\%} + 10)))]$

I1. $m = 0, L = []$

I2. Increment m

I3. $L.append((m, \Pr(R(q, n, t, m))))$

I4. If $m_{90\%}$ is not found Go to I2 else Return $(m_{90\%}, L)$

Fig. 4. $m_{90\%}(q, n, t)$ estimator

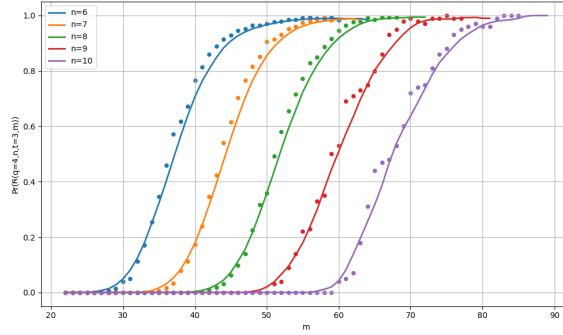


Fig. 5. $\Pr(R(q = 4, n, t = 3, m))$ for $n \in \{6, 7, 8, 9, 10\}$ (each point represents $\Pr(R(q = 4, n, t = 3, m))$ and the curves represent the interpolation of $\Pr(R(q = 4, n, t = 3, m))$)

R . Plotting L for $q = 4, t = 3, n \in \{6, 7, 8, 9, 10\}$ yields Figure 5. These curves suggest that $m_{90\%}(q = 4, n, t = 3)$ varies linearly with n . Based on five measured values of $m_{90\%}(q = 4, n, t = 3)$, we performed a linear regression

yielding

$$m_{90\%}(4, n, 3) = 8.10n - 5. \quad (13)$$

At $q = 4$ and $t = 3$, and targeting the specific parameters proposed in [6], we need to estimate $m_{90\%}$ $n \geq 13$. Eq. (13) leads to $m_{90\%}(4, 13, 3) = 100$. By adding 10 extra points as a safety margin (i.e., $m = 110$), we recover the solution in 96% in practice with $K = 50$.

For $q = 4$ and $t = 4$ with $n \in \{8, 9, 10\}$ and $K = 100$ the same procedure yields

$$m_{90\%}(4, n, 4) = 24.5n - 40.5 \quad (14)$$

so that $m_{90\%}(4, 13, 4) = 278$. Again, by adding 10 extra points as a safety margin (i.e., $m = 288$), we get a probability of success of 98% in practice with $K = 50$.

Equations (13) and (14) suggest the following conjecture

$$m_{90\%}(q, n, t) = (q - 1)^{t-1}n + C(q, t) \quad (15)$$

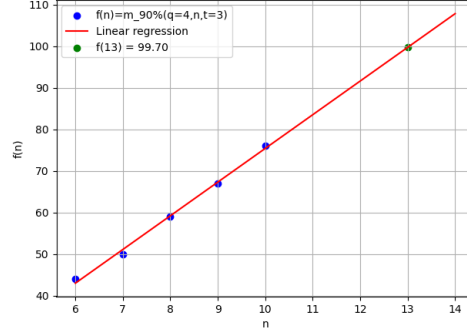


Fig. 6. $f(n) = m_{90\%}(q = 4, n, t = 3)$ for $n \in \{6, 7, 8, 9, 10\}$ where $f(6) = 44$, $f(7) = 50$, $f(8) = 59$, $f(9) = 67$, $f(10) = 76$ (each blue point represents $f(n)$ and the curve represent the linear regression of $f(n)$)

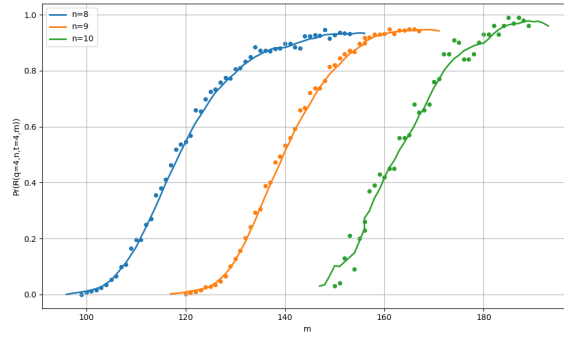


Fig. 7. $Pr(R(q = 4, n, t = 4, m))$ for $n \in \{8, 9, 10\}$ (each point represents $Pr(R(q = 4, n, t = 4, m))$ and the curves represent the interpolation of $Pr(R(q = 4, n, t = 4, m))$)

For some function C of q and t . We do not exactly know how this $C(q, t)$ behaves, but the estimations given by Equations (13) and (14) lead us to believe that $C(q, t) \leq 0$ for the values of q and t that are of interest to us. with $C(q, t) \in \mathbb{R}$. Estimating $m_{90\%}(q = 3, n, t)$ by choosing $C(q = 3, t = 3) = 0$, $C(q = 3, t = 4) = 0$ and the parameters from [7] requires $n = 22$, yielding $m_{90\%}(3, 22, 3) = 88$ and $m_{90\%}(q = 3, n = 22, t = 4) = 176$. In practice, this is even better. In both cases, we obtain a 100% of accuracy for the aforementioned values of $m_{90\%}$ with $K = 50$.

In summary, for any instance: if $q = 4$, depending on t (3 or 4) we estimate $m_{90\%}(q = 4, n, t)$ using equation (13) or (14), and for other values of q , we compute $m_{90\%}$ via equation (15) with $C(q, t) = 0$. Then, in any case, if we dispose of enough evaluation points, we choose a subset of $m = m_{90\%}(q = 4, n, t) + 10$ of them to run Algorithm I. Table 4 show T , the execution time, and M , RAM

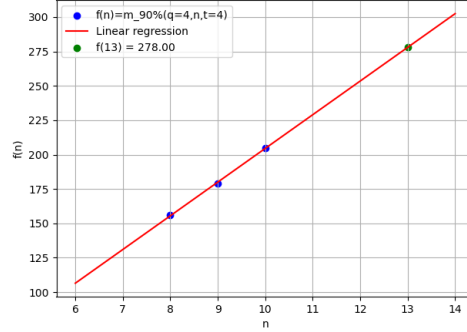


Fig. 8. $f(n) = m_{90\%}(q = 4, n, t = 4)$ for $n \in \{8, 9, 10\}$ where $f(8) = 156$, $f(9) = 179$, $f(10) = 205$ (each blue point represents $f(n)$ and the curve represent the linear regression of $f(n)$)

consumption for successful interpolations using the MOSEK solver across various values of q , n , t and m .

4 Attack implementation and results

In this section we give a summary of all the QA-SD instances from [7, 6, 22] for which our attack is successful.

4.1 Parameters from [7]

Two parameter sets are proposed in [7] with $q = 3$: $c = 2, t = 152$ and $c = 4, t = 64$. The suggested number of variables is $n = 25, 30, 25$. As hinted in [7], we assume the regular noise distribution for the e_i 's. These parameters were initially assumed to offer 128 bits of security. In [6], this estimate was revised to 118 bits because of so-called “folding attacks”. In fact, these parameters can be attacked in practice.

q	t	n	m	T	M
3	3	20	80	13min	33GB
3	3	21	84	27min	78GB
3	3	22	88	57min	163GB
3	4	20	160	26min	70GB
3	4	21	168	55min	147GB
3	4	22	176	1h57min	307GB

q	t	n	m	T	M
4	3	11	94	2min	8GB
4	3	12	102	7min	31GB
4	3	13	110	20min	111GB
4	4	11	239	5min	21GB
4	4	12	264	17min	93GB
4	4	13	288	56min	305GB

Table 4. Execution time T and RAM consumption M of a successful interpolation with the MOSEK solver

For the first one ($c = 2, t = 152$) with $n = 25$ (one of the suggested values), we can write the decomposition (5) on $n' = 3$ variables simultaneously, yielding a total of 8 sub-polynomials, and we bet that at least one of them has less than $t' = 4$ monomials (cf. Section 2.2). This decomposition also provide us with a total of 639 evaluation points for all the aforementioned sub-polynomials, which is far more than the $m = 176$ we need to interpolate 4-sparse polynomials or the $m = 352$ that we conjecture are needed to interpolate 5-sparse polynomials (cf. Section 3.6). We verified in practice that interpolating 4-sparse polynomials is feasible (see Table 4) and runs in 2 hours on a single 32-core machine. The process has to be repeated on all the 8 polynomials occurring in all 2300 subsets of 3 variables out of 25 (but can be parallelized). Table 5 shows the probability that e_0 splits with a sufficiently sparse sub-polynomial using eq.(5). It follows that there is a practical algorithm that recognizes QA-SD samples with probability at least 13.2% and runs in about 1M CPU.h. In theory, interpolating 5-sparse polynomials should be feasible and this would boost the recognizing probability to 42%. However we have not been able to verify this in practice because it would require too much RAM.

For the second parameter ($c = 4, t = 64$), with $n = 30$ we can interpolate (5) on $n' = 2$ variables simultaneously, obtaining 505 evaluation points for all 4 sub-polynomials. Agains this is sufficient to interpolate 4-sparse and 5-sparse polynomials. We have not been able to run the experiments (lack of RAM) but this would yield recognizers that work with probability 9.5% (resp. 31%) if 4-sparse interpolation is practically feasible (resp. 5-sparse).

We conclude that both parameter sets given in [7] are practically broken.

4.2 Parameters proposed in [6]

Two parameter sets have been proposed for $\mathbb{F}_4\text{OLEAGE}$, both with $q = 4, t = 27$ and $n \in \{14, 15, 16, 17, 18\}$: the “aggressive” one with $c = 3$ and the “conservative” one with $c = 4$.

t'	$p'(2, 25, 152, 3, t')$	$p'(2, 30, 64, 2, t')$	$p(3, 20, 33, 1, t')$	$p'(3, 21, 27, 1, t')$
0	0.00%	0.00%	0.01%	0.08%
1	0.02%	0.03%	0.15%	1.21%
2	0.32%	0.25%	1.22%	7.55%
3	2.17%	1.67%	6.13%	25.88%
4	10.65%	7.52%	19.66%	42.23%
5	29.54%	21.89%	37.09%	21.40%
6	38.71%	36.12%	29.57%	1.64%
7	17.11%	26.34%	6.04%	0.00%
8	1.47%	5.94%	0.13%	0.00%
9	0.01%	0.25%	0.00%	0.00%
10	0.00%	0.00%	0.00%	0.00%

Table 5. Probability that e_0 splits with a sub-polynomial of t' terms when writing eq. (5) for each possible subset of the variables, for the regular noise distribution.

Regarding the aggressive parameter set with $n = 14$, recall from Section 2.2 that e_0 can be evaluated on 298,935 points on expectation, and that writing the univariate decomposition (4) yields ≈ 389 evaluations of the 3 sub-polynomials. This is sufficient to interpolate 4-sparse polynomials according to the experiments shown in Section 3.6. The actual interpolation process itself takes one hour on 32 cores.

Table 2 shows that e_0 splits into sub-polynomials with at most 4 terms with probability 61%. We conclude that there is a recognizer that runs in 1056 CPU.h and that succeeds with probability 60%. We actually executed it in practice:

- We generated a random instance of the aggressive parameter set
- We ran the 0-, 1- and 2-sparse recognizers that are very efficient (this requires 1 minute on a single core). This discovered two monomials of e_0 (this had a 4.95% chance of happening, so we were a bit lucky). At this stage, we knew for sure that the input was a QA-SD sample, but we had not yet solved the search problem.
- After that, we ran the interpolation process for 4-sparse polynomials in all the 3×11 sub-polynomials in (4) for each possible variable $X_4, \dots, X_{14}$ (we excluded the first 3 tainted variables because of the regular noise distribution). Each interpolation took about an hour and required 300GB of RAM. This took several hours of wall-clock time using a cluster. This discovered 3 extra monomials.
- We ran the whole process again, and this time we recovered all the 27 monomials of e_0 .

The process succeeded on the first try (this had a 61% chance of happening). We have thus solved the search QA-SD problem in practice for “aggressive” instances of $\mathbb{F}_4\text{OLEAGE}$.

Regarding the conservative parameter set, the larger value of c restricts the number of available evaluation points. With $n = 14$, we would end up with just 6 evaluation points for each sub-polynomial on expectation. However, with $n = 18$ we would have 492 evaluation points and the exact same attack would be possible (although the space requirements are going to be 81 times larger). We did not try to execute it in practice, but we also claim that the conservative parameter set is practically insecure.

4.3 Parameters proposed in [22]

Twenty parameter sets have been proposed in [22] for various field sizes $q = 4, 8, 16$. Several are practically broken by the very efficient 0-, 1- and 2-sparse polynomial recognizers of section 2.2 and we actually executed the attack on several of them. For instance, consider $q = 4, c = 2, t = 42, n = 16$ with uniform noise. Using (5) for all 120 subsets of $n' = 2$ variables among 16, interpolating the 3×120 polynomials on the 18 expected evaluation points and detecting 0- and 1-sparse subpolynomials allows completely recovering e_0 in 15 hours on a single CPU core. The same process takes 24 hours for $q = 4, c = 2, t = 66, n = 17$

(which was expected to offer 128 bits of security). We tried to run the attack on $n = 19, t = 81$ and regular noise but ran out of patience after a week of sequential computation. The attack is nevertheless practically feasible.

The parameter sets with $q = 4, c = 4, t = 33, n = 18, 19, 20$ and uniform noise can also be practically broken. Using eq. (4) with a single variable, we obtain at least 492 evaluation points for each of the 3 sub-polynomials. This is sufficient to run the recognizer for 4-sparse polynomials that requires 486 points. Table 5 shows the probability that e_0 splits with a small sub-polynomial. The efficient recognizers for 0-, 1- and 2-sparse sub-polynomials could likely be run in practice (we have not tried), and would recognize QA-SD samples with probability 1.38%. Using the more sophisticated 4-sparse polynomial recognizer does not seem practically feasible because of the RAM requirements of the SOCP solver, but would recognize QA-SD instances with probability 27%.

The parameter set with $q = 4, c = 5, n = 21, t = 27$ is interesting. Writing the univariate decomposition of eq. (4) we obtain on average 207 evaluation points for each sub-polynomial. This is enough to interpolate 3-sparse polynomials according to section 3.6. We do not have enough RAM to test the process in practice, but the efficient recognizers of Section 2.2 for 0-, 1- and 2-sparse polynomials are still practical. If 3-sparse interpolation were practical, this would yield a recognizer with success probability 34.7%. Otherwise, there is always a practical recognizer with success probability 8.84%.

We highlight the fact that there are some parameter sets to which our attack does not apply because we do not have enough evaluation points of e_0 to obtain the univariate decomposition of eq. (4) in the first place: $q = 4, c = 9, t = 9, n = 19$; $q = 8, c = 3, t = 49, n = 12$; all parameter sets with $q = 16$.

5 Potential countermeasures and future work

There are several simple ways to avoid our attacks: increase t or (even better) increase c . Indeed, if $(q - 1)^{n-1} < q^{(q-1)(c-1)}$, then the adversary does not even have a single evaluation point for e_0 . So, choosing $c \geq 1 + (n - 1)/(q - 1)$ is sufficient to avoid our attack. This also suggests using larger field sizes, using the “trace” technique of [22].

However, increasing either c or t has an adverse effect on the performance of the resulting scheme, with communication of order $(ct)^2$. However, c and t play asymmetrical roles in the security of the scheme (because of our attacks), a fact that was not previously well-understood.

An interesting alternative would be to sample regular public polynomials a_i , namely such that $a_i(\mathbf{x}) \neq 0$. This modifies the definition of the problem, but this would prevent the adversary from evaluating e_0 . However, the adversary could then target points x such that $a_1(\mathbf{x}) = \dots = a_c(\mathbf{x}) = 1$ and obtain evaluations of $e_0 + e_1 + \dots + e_c$. This is a ct -sparse polynomial and it will be harder to interpolate.

From the cryptanalysis side, an obvious potential development would consist in reducing the memory complexity of the attack following the lines of Section 3.5 to make it even more practical.

Acknowledgments. We are thankful to Maxime Bombar, Clément Ducros and Dung Bui for interesting discussions about $\mathbb{F}_4$ OLEAGE. We acknowledge financial support from the French *Agence Nationale de la Recherche* under project “GORILLA” (ANR-20-CE39-0002), “PostCryptum” (ANR20-ASTR-0011), “SANGRIA” (ANR-21-CE39-0006) and “CRYPTANALYSE” (ANR-22-PECY-0010). Experiments presented in this paper were carried out using the Grid’5000 testbed, supported by a scientific interest group hosted by Inria and including CNRS, RENATER and several Universities as well as other organizations. This work was also granted access to HPC resources of “Convergence@Paris” at LIP6, Sorbonne Université. “Convergence@Paris” is co-financed by the Région Île-de-France, Sorbonne Université, LIP6, IRIF and LCQB among others.

References

1. Erling D. Andersen, Cornelis Roos, and Tamás Terlaky. On implementing a primal-dual interior-point method for conic quadratic optimization. *Math. Program.*, 95(2):249–277, 2003.
2. MOSEK ApS. *The MOSEK Optimizer API for Python 11.0.20.*, 2025.
3. Andrew Arnold and Daniel S. Roche. Multivariate sparse interpolation using randomized kronecker substitutions. In Katsusuke Nabeshima, Kosaku Nagasaka, Franz Winkler, and Ágnes Szántó, editors, *International Symposium on Symbolic and Algebraic Computation, ISSAC ’14, Kobe, Japan, July 23-25, 2014*, pages 35–42. ACM, 2014.
4. Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation (extended abstract). In *20th ACM STOC*, pages 1–10. ACM Press, May 1988.
5. Michael Ben-Or and Prasoona Tiwari. A deterministic algorithm for sparse multivariate polynomial interpolation (extended abstract). In Janos Simon, editor, *Proceedings of the 20th Annual ACM Symposium on Theory of Computing, May 2-4, 1988, Chicago, Illinois, USA*, pages 301–309. ACM, 1988.
6. Maxime Bombar, Dung Bui, Geoffroy Couteau, Alain Couvreur, Clément Ducros, and Sacha Servan-Schreiber. FOLEAGE: $\mathbb{F}_4$ OLE-based multi-party computation for boolean circuits. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part VI*, volume 15489 of *LNCS*, pages 69–101. Springer, Singapore, December 2024.
7. Maxime Bombar, Geoffroy Couteau, Alain Couvreur, and Clément Ducros. Correlated pseudorandomness from the hardness of quasi-abelian decoding. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 567–601. Springer, Cham, August 2023.
8. Marco Bonetti, Pasquale Cirillo, and Anton Ogay. Computing the exact distributions of some functions of the ordered multinomial counts: maximum, minimum, range and sums of order statistics. *Royal Society Open Science*, 6(10):190198, 2019.
9. Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

10. Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 489–518. Springer, Cham, August 2019.
11. Emmanuel J. Candès, Justin K. Romberg, and Terence Tao. Robust uncertainty principles: exact signal reconstruction from highly incomplete frequency information. *IEEE Trans. Inf. Theory*, 52(2):489–509, 2006.
12. Emmanuel J. Candès and Terence Tao. Near-optimal signal recovery from random projections: Universal encoding strategies? *IEEE Trans. Inf. Theory*, 52(12):5406–5425, 2006.
13. Emmanuel J. Candès, Justin K. Romberg, and Terence Tao. Stable signal recovery from incomplete and inaccurate measurements. *Communications on Pure and Applied Mathematics: A Journal Issued by the Courant Institute of Mathematical Sciences*, 59(8):1207–1223, 2006.
14. Charles J. Corrado. The exact distribution of the maximum, minimum and the range of multinomial/dirichlet and multivariate hypergeometric frequencies. *Statistics and Computing*, 21(3):349–359, Jul 2011.
15. Etienne de Klerk and Dmitrii V. Pasechnik. Approximation of the stability number of a graph via copositive programming. *SIAM J. Optim.*, 12(4):875–892, 2002.
16. David L. Donoho. Compressed sensing. *IEEE Trans. Inf. Theory*, 52(4):1289–1306, 2006.
17. Mark Giesbrecht and Daniel S. Roche. Diversification improves interpolation. In Éric Schost and Ioannis Z. Emiris, editors, *Symbolic and Algebraic Computation, International Symposium, ISSAC 2011 (co-located with FCRC 2011), San Jose, CA, USA, June 7-11, 2011, Proceedings*, pages 123–130. ACM, 2011.
18. Ming-Deh A. Huang and Ashwin J. Rao. Interpolation of sparse multivariate polynomials over large finite fields with applications. In Éva Tardos, editor, *Proceedings of the Seventh Annual ACM-SIAM Symposium on Discrete Algorithms, 28-30 January 1996, Atlanta, Georgia, USA*, pages 508–517. ACM/SIAM, 1996.
19. Qiao-Long Huang and Xiao-Shan Gao. Sparse polynomial interpolation with finitely many values for the coefficients. In Vladimir P. Gerdt, Wolfram Koepf, Werner M. Seiler, and Evgenii V. Vorozhtsov, editors, *Computer Algebra in Scientific Computing - 19th International Workshop, CASC 2017, Beijing, China, September 18-22, 2017, Proceedings*, volume 10490 of *Lecture Notes in Computer Science*, pages 196–209. Springer, 2017.
20. Kenneth Ireland and Michael Rosen. *A classical introduction to modern number theory.*, volume 84 of *Grad. Texts Math.* New York etc.: Springer-Verlag, 2nd ed. edition, 1990.
21. Erich L. Kaltofen, Yagati N. Lakshman, and J.-M. Wiley. Modular rational sparse multivariate polynomial interpolation. In Shunro Watanabe and Morio Nagata, editors, *Proceedings of the International Symposium on Symbolic and Algebraic Computation, ISSAC '90, Tokyo, Japan, August 20-24, 1990*, pages 135–139. ACM, 1990.
22. Zhe Li, Chaoping Xing, Yizhou Yao, and Chen Yuan. Efficient pseudorandom correlation generators for any finite field. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part V*, volume 15605 of *Lecture Notes in Computer Science*, pages 145–175. Springer, 2025.
23. Rudolf Lidl and Harald Niederreiter. *Finite fields.*, volume 20 of *Encycl. Math. Appl.* Cambridge: Cambridge Univ. Press, 2nd ed. edition, 1996.

24. Yurii E. Nesterov and Arkadii Nemirovskii. *Interior-point polynomial algorithms in convex programming*, volume 13 of *Siam studies in applied mathematics*. SIAM, 1994.
25. Madhu Sudan. Maximum likelihood decoding of reed solomon codes. In *37th Annual Symposium on Foundations of Computer Science, FOCS '96, Burlington, Vermont, USA, 14-16 October, 1996*, pages 164–172. IEEE Computer Society, 1996.
26. Joris van der Hoeven and Grégoire Lecerf. Sparse polynomial interpolation: faster strategies over finite fields. *Applicable Algebra in Engineering, Communication and Computing*, 2024.
27. Hal Wasserman. Reconstructing randomly sampled multivariate polynomials from highly noisy data. In Howard J. Karloff, editor, *Proceedings of the Ninth Annual ACM-SIAM Symposium on Discrete Algorithms, 25-27 January 1998, San Francisco, California, USA*, pages 59–67. ACM/SIAM, 1998.
28. Richard Zippel. Probabilistic algorithms for sparse polynomials. In Edward W. Ng, editor, *Symbolic and Algebraic Computation, EUROSAM '79, An International Symposium on Symbolic and Algebraic Computation, Marseille, France, June 1979, Proceedings*, volume 72 of *Lecture Notes in Computer Science*, pages 216–226. Springer, 1979.

Supplementary Material

A Proof of Theorem 2

We first recall the statement of Theorem 2: We consider the ring of n -variate polynomials $\mathbb{F}_q[X_1, \dots, X_n]$ over $\mathbb{F}_q$ and $P \in \mathbb{F}_q[X_1, \dots, X_n]$ a polynomial with $t \geq 1$ monomials. We denote

$$P(X_1, \dots, X_n) = P(\mathbf{X}) = \sum_{j=1}^t u_j X_1^{\alpha_{j,1}} \dots X_n^{\alpha_{j,n}} = \sum_{j=1}^t u_j \mathbf{X}^{\alpha_j} \quad (16)$$

where $\alpha_j = (\alpha_{j,1}, \dots, \alpha_{j,n})$ is a vector of integers in $\{0, \dots, d-1\}$ and $u_j \in \mathbb{F}_q^*$ for all $j \in \{1, \dots, t\}$.

Theorem 2. *Let q be a prime power and $n, t \geq 1$ be two integers. Let χ be a primitive multiplicative character of $\mathbb{F}_q$. Let $P \in \mathbb{F}_q[X_1, \dots, X_n]$ a polynomial with $t \geq 1$ monomials. There exists a polynomial $F \in \mathbb{C}[Z_1, \dots, Z_n]$ with at most d^{t-1} monomials such that for all $(\gamma_1, \dots, \gamma_n) \in (\mathbb{F}_q^*)^n$,*

$$F(\chi(\gamma_1), \dots, \chi(\gamma_n)) = \chi(P(\gamma_1, \dots, \gamma_n)).$$

Moreover, if P is defined as in (16), we have

$$F(\mathbf{Z}) = F(Z_1, \dots, Z_n) = \sum_{\mathbf{i} \in \{0, \dots, d-1\}^t} (c_{\mathbf{i}} \chi(u_1^{i_1} \dots u_t^{i_t})) \cdot \mathbf{Z}^{i_1 \alpha_1 + \dots + i_t \alpha_t \bmod d}$$

where the coefficients $c_{\mathbf{i}}$ satisfy the following conditions:

1. $c_{\mathbf{i}} = 0$ if $i_1 + \dots + i_t \not\equiv 1 \pmod{d}$;
2. $c_{\mathbf{i}} = (-1/d)^{t-1}$ if $i_l = 1$ for some $l \in \{1, \dots, t\}$, and $i_k = 0$ for $k \in \{1, \dots, t\} \setminus \{l\}$;
3. $|c_{\mathbf{i}}| > 1/d^{t-1}$ otherwise.

As mentioned in the core of the paper, its proof relies on simple computation and well-known facts on Gauss and Jacobi sums (see e.g., [20, Chapter 8] or [23, Chapter 5]). For the reader's convenience, we include in this appendix a detailed and self-contained proof, along with all necessary preliminaries.

Characters. Let $\mathbb{G}$ be a finite Abelian group of order n . A *character* of $\mathbb{G}$ is a group homomorphism $\chi : \mathbb{G} \rightarrow \mathbb{C}^*$, where $\mathbb{C}^*$ denotes the multiplicative group of nonzero complex numbers. The set of all characters of $\mathbb{G}$, denoted by $\widehat{\mathbb{G}}$, forms an Abelian group under pointwise multiplication (called $\mathbb{G}$'s dual group):

$$(\chi \cdot \psi)(g) = \chi(g)\psi(g) \quad \text{for all } g \in \mathbb{G}.$$

The image of any character $\chi \in \widehat{\mathbb{G}}$ is contained in the group of n -th roots of unity in $\mathbb{C}^*$. The space $\mathbb{C}[\mathbb{G}]$ of complex-valued functions on $\mathbb{G}$ is a Hermitian inner product space with scalar product defined by

$$\langle f, g \rangle = \frac{1}{|\mathbb{G}|} \sum_{x \in \mathbb{G}} f(x) \overline{g(x)}.$$

The set $\widehat{\mathbb{G}}$ forms an orthonormal basis of $\mathbb{C}[\mathbb{G}]$ with respect to this inner product. This orthonormality allows one to define the *discrete Fourier transform* on $\mathbb{G}$ as the linear map

$$\widehat{f} : \widehat{\mathbb{G}} \longrightarrow \mathbb{C}, \quad \widehat{f}(\chi) = \langle f, \chi \rangle = \frac{1}{|\mathbb{G}|} \sum_{x \in \mathbb{G}} f(x) \overline{\chi(x)}.$$

The inverse transform is given by the Fourier inversion formula:

$$f(x) = \sum_{\chi \in \widehat{\mathbb{G}}} \widehat{f}(\chi) \chi(x).$$

A *multiplicative character* of $(\mathbb{F}_q)$ is a character of the multiplicative group $(\mathbb{F}_q^*, \cdot)$, $\chi : \mathbb{F}_q^* \rightarrow \mathbb{C}^*$, extended to $\mathbb{F}_q$ by setting $\chi(0) = 0$. An *additive character* $\psi : \mathbb{F}_q \rightarrow \mathbb{C}^\times$ of $(\mathbb{F}_q)$ is a character of the additive group $(\mathbb{F}_q, +)$.

Multiplicative characters of $(\mathbb{F}_q^)^t$.* Let q be a prime power and $t \geq 1$ be an integer. The *multiplicative characters* of $(\mathbb{F}_q^*)^t$ are the characters of the multiplicative group $(\mathbb{F}_q^*)^t$ into $\mathbb{C}^*$. (where the group law on $(\mathbb{F}_q^*)^t$ is componentwise multiplication). Since the dual group $\widehat{(\mathbb{F}_q^*)^t}$ is isomorphic to $(\mathbb{F}_q^*)^t$ itself, we have a natural isomorphism $\widehat{(\mathbb{F}_q^*)^t} \cong (\mathbb{F}_q^*)^t$. and, in particular, each character $\chi \in \widehat{(\mathbb{F}_q^*)^t}$ can be written uniquely as a product

$$\chi(\theta_1, \dots, \theta_t) = \chi_1(\theta_1) \cdots \chi_t(\theta_t)$$

where each χ_j is a multiplicative character of $\mathbb{F}_q$, that is, an element of $\widehat{\mathbb{F}_q^*}$.

Gauss and Jacobi sums. Given a multiplicative character χ and an additive character ψ of $\mathbb{F}_q$, the *Gauss sum* $G(\chi, \psi)$ is defined by

$$G(\chi, \psi) = \sum_{x \in \mathbb{F}_q} \chi(x) \psi(x).$$

It is well known (e.g. , [23, Theorem 5.11]) that if χ and ψ are nontrivial, then $|G(\chi, \psi)| = \sqrt{q}$.

Let $M \geq 2$ be an integer. Let $\chi_1, \dots, \chi_M$ be multiplicative characters on $\mathbb{F}_q$, and $s \in \mathbb{F}_q$. The *Jacobi sum* $J_s(\chi_1, \dots, \chi_M)$ is defined by:

$$J_s(\chi_1, \dots, \chi_M) = \sum_{\substack{x_1 + \dots + x_M = s \\ x_1, \dots, x_M \in \mathbb{F}_q}} \chi_1(x_1) \cdots \chi_M(x_M).$$

If all χ_i are trivial multiplicative characters then $J_s(\chi_1, \dots, \chi_M) = q^{M-1}$ for all $s \in \mathbb{F}_q$ [23, Theorem 5.19]. If some χ_i is trivial (but not all of them), then $J_s(\chi_1, \dots, \chi_M) = 0$ for all $s \in \mathbb{F}_q$ [23, Theorem 5.19]. If all χ_i 's are nontrivial multiplicative characters, then

$$J_1(\chi_1, \dots, \chi_M) = \frac{G(\chi_1, \psi) \cdots G(\chi_M, \psi)}{G(\chi_1 \cdots \chi_M, \psi)}.$$

if their product $\chi_1 \cdots \chi_M$ is nontrivial, and

$$J_1(\chi_1, \dots, \chi_M) = -\frac{1}{q} G(\chi_1, \psi) \cdots G(\chi_M, \psi).$$

otherwise (for any nontrivial additive character ψ). [23, Theorem 5.21]. Finally if χ_M is nontrivial, then $J_0(\chi_1, \dots, \chi_M) = 0$ if $\chi_1 \cdots \chi_M$ is the trivial character and $J_0(\chi_1, \dots, \chi_M) = \chi_M(-1) \cdot (q-1) \cdot J_1(\chi_1, \dots, \chi_{M-1})$ otherwise [23, Theorem 5.20].

The proof of Theorem 2 makes use of this simple combinatorial lemma:

Lemma 2. *Let $M \geq 1$ be an integer and $A_M(q)$ denote the number of vectors $(\theta_1, \dots, \theta_M) \in (\mathbb{F}_q^*)^M$ such that $\theta_1 + \cdots + \theta_M = 0$. We have*

$$A_M(q) = \frac{1}{q} ((q-1)^M + (-1)^M (q-1)).$$

Proof. To prove this lemma, we consider $\xi : \mathbb{F}_q \rightarrow \mathbb{C}^*$ a nontrivial *additive* character of $\mathbb{F}_q$. By the orthogonality of additive characters [23, Theorem 5.4]:

$$\frac{1}{q} \sum_{x \in \mathbb{F}_q} \xi(ax) = \begin{cases} 1 & \text{if } a = 0, \\ 0 & \text{if } a \neq 0. \end{cases} \quad (17)$$

We can thus write the indicator function for the condition “ $\theta_1 + \cdots + \theta_M = 0$ ” as:

$$\begin{aligned} A_M(q) &= \sum_{(\theta_1, \dots, \theta_M) \in (\mathbb{F}_q^*)^M} \frac{1}{q} \sum_{x \in \mathbb{F}_q} \xi(x(\theta_1 + \cdots + \theta_M)) \\ &= \frac{1}{q} \sum_{x \in \mathbb{F}_q} \sum_{(\theta_1, \dots, \theta_M) \in (\mathbb{F}_q^*)^M} \xi(x(\theta_1 + \cdots + \theta_M)) \\ &= \frac{1}{q} \sum_{x \in \mathbb{F}_q} \prod_{i=1}^M \left(\sum_{\theta_i \in \mathbb{F}_q^*} \xi(x\theta_i) \right) = \frac{1}{q} \sum_{x \in \mathbb{F}_q} \left(\sum_{\theta \in \mathbb{F}_q^*} \xi(x\theta) \right)^M. \end{aligned}$$

By (17), we have

$$\sum_{\theta \in \mathbb{F}_q^*} \xi(x\theta) = \begin{cases} q-1 & \text{if } x = 0, \\ -1 & \text{if } x \neq 0, \end{cases}$$

and

$$A_{t-1}(q) = \frac{1}{q} ((q-1)^M + (q-1)(-1)^M)$$

as desired.

Let $\ell \geq 1$ be an integer and let $\mathbf{i} = (i_1, \dots, i_\ell)$ be a ℓ -dimensional vector of integers in $\{0, \dots, d-1\}$. Theorem 2's proof involves the following sums of characters

$$S_{\ell, \mathbf{i}}(s) = \sum_{\theta_1, \dots, \theta_\ell \in \mathbb{F}_q^*} \chi(\theta_1 + \dots + \theta_\ell + s) \chi^{d-i_1}(\theta_1) \dots \chi^{d-i_\ell}(\theta_\ell)$$

for $s \in \mathbb{F}_q$ and $i_1 + \dots + i_\ell = 1 \pmod{d}$

Lemma 3. *We have*

1. $S_{1, \mathbf{i}}(0) = q - 1$;
2. $|S_{\ell, \mathbf{i}}(0)| = q^{(\ell-1)/2}(q-1)$ for all $\ell \geq 1$.

Proof.

1. For $\ell = 1$, we have necessarily $\mathbf{i} = (1)$ and a direct computation shows:

$$S_{1, \mathbf{i}}(0) = \sum_{\theta \in \mathbb{F}_q^*} \chi(\theta) \cdot \chi^{d-1}(\theta) = \sum_{\theta \in \mathbb{F}_q^*} \chi^d(\theta) = \sum_{\theta \in \mathbb{F}_q^*} 1 = (q-1).$$

2. We have

$$S_{\ell, \mathbf{i}}(0) = \sum_{\theta_1, \dots, \theta_\ell \in \mathbb{F}_q^*} \chi(\theta_1 + \dots + \theta_\ell) \chi^{d-i_1}(\theta_1) \dots \chi^{d-i_\ell}(\theta_\ell)$$

We introduce a variable $\theta_{\ell+1} = -(\theta_1 + \dots + \theta_\ell)$ such that $\theta_1 + \dots + \theta_{\ell+1} = 0$ and the sum can be rewritten as

$$S_{\ell, \mathbf{i}}(0) = \sum_{\substack{\theta_1 + \dots + \theta_{\ell+1} = 0 \\ \theta_1, \dots, \theta_{\ell+1} \in \mathbb{F}_q}} \chi(-\theta_{\ell+1}) \chi^{d-i_1}(\theta_1) \dots \chi^{d-i_\ell}(\theta_\ell)$$

since $\chi(0) = 0$. We thus have

$$\begin{aligned} S_{\ell, \mathbf{i}}(0) &= \chi(-1) \sum_{\substack{\theta_1 + \dots + \theta_{\ell+1} = 0 \\ \theta_1, \dots, \theta_{\ell+1} \in \mathbb{F}_q}} \chi(\theta_{\ell+1}) \chi^{d-i_1}(\theta_1) \dots \chi^{d-i_\ell}(\theta_\ell) \\ &= \chi(-1) J_0(\chi, \chi_{d-i_1}, \dots, \chi_{d-i_\ell}). \end{aligned}$$

By assumption $i_1 + \dots + i_\ell = 1 \pmod{d}$, and we have $\chi^{d-i_1} \dots \chi^{d-i_\ell} = \chi^{-1}$ and $\chi \cdot \chi^{d-i_1} \dots \chi^{d-i_\ell} = \text{id}$

$$\begin{aligned} |S_{\ell, \mathbf{i}}(0)| &= |J_0(\chi, \chi_{d-i_1}, \dots, \chi_{d-i_\ell})| = (q-1) |J_1(\chi_{d-i_1}, \dots, \chi_{d-i_\ell})| \\ &= (q-1) \left| \frac{G(\chi_{d-i_1}, \psi) \dots G(\chi_{d-i_\ell}, \psi)}{G(\chi^{-1}, \psi)} \right| \\ &= (q-1) \frac{\sqrt{q}^\ell}{\sqrt{q}} = q^{(\ell-1)/2}(q-1). \end{aligned}$$

Lemma 4. *We have*

1. $S_{\ell, \mathbf{i}}(s) = S_{\ell, \mathbf{i}}(1)$ for all $s \in \mathbb{F}_q^*$ and all $\ell \geq 1$,
2. $S_{1, \mathbf{i}}(1) = -1$;
3. $|S_{\ell, \mathbf{i}}(1)| = q^{(\ell-1)/2}$ for all $\ell \geq 1$.

Proof.

1. Let $s \in \mathbb{F}_q^*$. We perform the change of variable $\theta_i = st_i$, where $t_i \in \mathbb{F}_q^*$. This is a bijection from $(\mathbb{F}_q^*)^\ell$ to itself, so:

$$S_{\ell, \mathbf{i}}(s) = \sum_{(t_1, \dots, t_\ell) \in (\mathbb{F}_q^*)^\ell} \chi(st_1 + \dots + st_\ell + s) \cdot \chi^{d-i_1}(st_1) \dots \chi^{d-i_\ell}(st_\ell).$$

By multiplicativity of χ , we have:

$$\begin{aligned} S_{\ell, \mathbf{i}}(s) &= \sum_{(\theta_1, \dots, \theta_\ell) \in (\mathbb{F}_q^*)^\ell} \chi(s) \chi(\theta_1 + \dots + \theta_\ell + 1) \\ &\quad \cdot \chi^{d-i_1}(s) \chi^{d-i_1}(\theta_1) \dots \chi^{d-i_\ell}(s) \chi^{d-i_\ell}(\theta_\ell) \\ &= \chi(s)^{1+\ell d - (i_1 + \dots + i_\ell)} \\ &\quad \cdot \sum_{(\theta_1, \dots, \theta_\ell) \in (\mathbb{F}_q^*)^\ell} \chi(\theta_1 + \dots + \theta_\ell + 1) \cdot \chi^{d-i_1}(\theta_1) \dots \chi^{d-i_\ell}(\theta_\ell). \end{aligned}$$

and thus $S_{\ell, \mathbf{i}}(s) = S_{\ell, \mathbf{i}}(1)$ for all $s \in \mathbb{F}_q^*$.

2. For $\ell = 1$, we have necessarily $\mathbf{i} = (1)$ and we observe that:

$$S_1(1) = \sum_{\theta \in \mathbb{F}_q^*} \chi\left(\frac{\theta+1}{\theta}\right) = \sum_{\theta \in \mathbb{F}_q^*} \chi\left(1 + \frac{1}{\theta}\right)$$

By the change of variable $u = 1/\theta$, which is a bijection on $\mathbb{F}_q^*$, we get

$$S_1(1) = \sum_{u \in \mathbb{F}_q^*} \chi(1+u).$$

Since χ is nontrivial, by orthogonality of characters, the sum for u over all $\mathbb{F}_q$ of $\chi(1+u)$ is zero, and thus $S_1(1) + \chi(1) = 0$ and $S_1(1) = -1$.

3. We have

$$S_{\ell, \mathbf{i}}(1) = \sum_{\theta_1, \dots, \theta_\ell \in \mathbb{F}_q^*} \chi(\theta_1 + \dots + \theta_\ell + 1) \chi^{d-i_1}(\theta_1) \dots \chi^{d-i_\ell}(\theta_\ell)$$

We introduce a variable $\theta_{\ell+1} = -(\theta_1 + \dots + \theta_\ell + 1)$ such that $\theta_1 + \dots + \theta_{\ell+1} = -1$ and the sum can be rewritten as

$$S_{\ell, \mathbf{i}}(1) = \sum_{\substack{\theta_1 + \dots + \theta_{\ell+1} = -1 \\ \theta_1, \dots, \theta_{\ell+1} \in \mathbb{F}_q}} \chi(-\theta_{\ell+1}) \chi^{d-i_1}(\theta_1) \dots \chi^{d-i_\ell}(\theta_\ell)$$

since $\chi(0) = 0$. We thus have

$$\begin{aligned} S_{\ell, \mathbf{i}}(1) &= \chi(-1) \sum_{\substack{\theta_1 + \dots + \theta_{\ell+1} = -1 \\ \theta_1, \dots, \theta_{\ell+1} \in \mathbb{F}_q}} \chi(\theta_{\ell+1}) \chi^{d-i_1}(\theta_1) \dots \chi^{d-i_\ell}(\theta_\ell) \\ &= \chi(-1) J_{-1}(\chi, \chi_{d-i_1}, \dots, \chi_{d-i_\ell}). \end{aligned}$$

By assumption $i_1 + \dots + i_\ell = 1 \pmod{d}$, and we have $\chi \cdot \chi^{d-i_1} \dots \chi^{d-i_\ell} = \text{id}$

$$\begin{aligned} |S_{\ell, \mathbf{i}}(1)| &= |J_{-1}(\chi, \chi_{d-i_1}, \dots, \chi_{d-i_\ell})| \\ &= \left| -\frac{1}{q} G(\chi, \psi) \cdot G(\chi_{d-i_1}, \psi) \dots G(\chi_{d-i_\ell}, \psi) \right| \\ &= \frac{\sqrt{q}^{\ell+1}}{q} = q^{(\ell-1)/2}. \end{aligned}$$

Proof (Theorem 2). Consider the function defined by $\psi(\theta_1, \dots, \theta_t) = \chi(\theta_1 + \dots + \theta_t)$ from $(\mathbb{F}_q^*)^t$ to $\mathbb{C}$. As mentioned above, it can be written as a sum with complex coefficients of basis terms from $(\mathbb{F}_q^*)^t$. Thus, as a sum of at most d^t terms. Let us denote

$$\psi(\theta_1, \dots, \theta_t) = \sum_{\mathbf{i} \in \{0, \dots, d-1\}^t} c_{\mathbf{i}} \chi^{i_1}(\theta_1) \dots \chi^{i_t}(\theta_t)$$

If we denote

$$F(\mathbf{Z}) = F(Z_1, \dots, Z_n) = \sum_{\mathbf{i} \in \{0, \dots, d-1\}^t} (c_{\mathbf{i}} \chi(u_1^{i_1} \dots u_t^{i_t})) \cdot \mathbf{Z}^{\alpha_1 i_1 + \dots + \alpha_t i_t \pmod{d}}$$

we have for all $\gamma = (\gamma_1, \dots, \gamma_n) \in (\mathbb{F}_q^*)^n$,

$$\begin{aligned} \chi(P(\gamma_1, \dots, \gamma_n)) &= \chi\left(\sum_{i=1}^t u_i \gamma^{\alpha_i}\right) \\ &= \psi(u_1 \gamma^{\alpha_1}, \dots, u_t \gamma^{\alpha_t}) \\ &= \sum_{\mathbf{i} \in \{0, \dots, d-1\}^t} c_{\mathbf{i}} \chi^{i_1}(u_1 \gamma^{\alpha_1}) \dots \chi^{i_t}(u_t \gamma^{\alpha_t}) \\ &= \sum_{\mathbf{i} \in \{0, \dots, d-1\}^t} (c_{\mathbf{i}} \chi(u_1^{i_1} \dots u_t^{i_t})) \chi(\gamma^{\alpha_1 \cdot i_1 + \dots + \alpha_t \cdot i_t}) \\ &= \sum_{\mathbf{i} \in \{0, \dots, d-1\}^t} (c_{\mathbf{i}} \chi(u_1^{i_1} \dots u_t^{i_t})) \chi(\gamma)^{\alpha_1 \cdot i_1 + \dots + \alpha_t \cdot i_t} \\ &= F(\chi(\gamma)) \end{aligned}$$

We have thus shown the existence of $F \in \mathbb{C}[Z_1, \dots, Z_n]$ with at most d^t monomials such that for all $\gamma \in (\mathbb{F}_q^*)^n$, $F(\chi(\gamma)) = \chi(P(\gamma))$. We will now prove the three statements on the coefficients $c_{\mathbf{i}}$ for $\mathbf{i} \in \{0, \dots, d-1\}^t$ (and in particular the first one will prove that F has at most d^{t-1} monomials).

Case $i_1 + \dots + i_t \neq 1 \pmod{d}$. For any $\lambda \in \mathbb{F}_q^*$ we have:

$$\psi(\lambda\theta_1, \dots, \lambda\theta_t) = \sum_{\mathbf{i} \in \{0, \dots, d-1\}^t} c_{\mathbf{i}} \chi^{i_1}(\lambda\theta_1) \cdots \chi^{i_t}(\lambda\theta_t)$$

and

$$\psi(\lambda\theta_1, \dots, \lambda\theta_t) = \chi(\lambda\theta_1 + \dots + \lambda\theta_t) = \chi(\lambda(\theta_1 + \dots + \theta_t)) = \chi(\lambda)\psi(\theta_1, \dots, \theta_t).$$

So a monomial associated with $\mathbf{i} = (i_1, \dots, i_t) \in \{0, \dots, d-1\}^t$ appears only if

$$\chi^{i_1}(\lambda\theta_1) \cdots \chi^{i_t}(\lambda\theta_t) = \chi(\lambda)\chi^{i_1}(\theta_1) \cdots \chi^{i_t}(\theta_t)$$

for all $(\theta_1, \dots, \theta_t) \in (\mathbb{F}_q^*)^t$, and hence only if

$$\chi(\lambda)^{i_1 + \dots + i_t} \cdot \chi^{i_1}(\theta_1) \cdots \chi^{i_t}(\theta_t) = \chi(\lambda)\chi^{i_1}(\theta_1) \cdots \chi^{i_t}(\theta_t)$$

for all $(\theta_1, \dots, \theta_t) \in (\mathbb{F}_q^*)^t$, that is, if and only if $i_1 + \dots + i_t \equiv 1 \pmod{d}$. Therefore, the only elements of $(\widehat{\mathbb{F}_q^*})^t$ that appear must be those with “weight” congruent to 1 mod d , of which there are d^{t-1} .

Case where there exists $j \in \{1, \dots, t\}$ s.t. $i_j = 1$ and $i_k = 0$ for $k \in \{1, \dots, t\} \setminus \{j\}$. Without loss of generality, we consider in the following the vector $\mathbf{i} = (1, 0, \dots, 0)$, i.e. with $i_1 = 1$ and $i_2 = \dots = i_t = 0$ (the other cases being similar). By the Fourier inversion formula, we have

$$c_{(1,0,\dots,0)} = \frac{1}{d^t} \sum_{\theta \in (\mathbb{F}_q^*)^t} \chi(\theta_1 + \dots + \theta_t) \cdot \overline{\chi(\theta_1)} = \frac{1}{d^t} \sum_{\theta \in (\mathbb{F}_q^*)^t} \chi(\theta_1 + \dots + \theta_t) \cdot \chi(\theta_1)^{d-1}.$$

This sum is not a Jacobi sum but can be written as a sum of Jacobi sums depending on the value of the sum $\theta_2 + \dots + \theta_t$. Indeed, we have

$$\begin{aligned} c_{(1,0,\dots,0)} &= \frac{1}{d^t} \sum_{\theta_1 \in (\mathbb{F}_q^*)} \sum_{(\theta_2, \dots, \theta_t) \in (\mathbb{F}_q^*)^{t-1}} \chi(\theta_1 + \dots + \theta_t) \cdot \chi(\theta_1)^{d-1} \\ &= \frac{1}{d^t} \sum_{\theta_1 \in (\mathbb{F}_q^*)} \sum_{s \in \mathbb{F}_q} \sum_{\substack{(\theta_2, \dots, \theta_t) \in (\mathbb{F}_q^*)^{t-1} \\ \theta_2 + \dots + \theta_t = s}} \chi(\theta_1 + \dots + \theta_t) \cdot \chi(\theta_1)^{d-1} \\ &= \frac{1}{d^t} \sum_{\theta_1 \in (\mathbb{F}_q^*)} \sum_{s \in \mathbb{F}_q} \sum_{\substack{(\theta_2, \dots, \theta_t) \in (\mathbb{F}_q^*)^{t-1} \\ \theta_2 + \dots + \theta_t = s}} \chi(\theta_1 + s) \cdot \chi(\theta_1)^{d-1} \\ &= \frac{1}{d^t} \sum_{s \in \mathbb{F}_q} \sum_{\substack{(\theta_2, \dots, \theta_t) \in (\mathbb{F}_q^*)^{t-1} \\ \theta_2 + \dots + \theta_t = s}} \sum_{\theta_1 \in (\mathbb{F}_q^*)} \chi(\theta_1 + s) \cdot \chi(\theta_1)^{d-1} \\ &= \frac{1}{d^t} \sum_{s \in \mathbb{F}_q} \sum_{\substack{(\theta_2, \dots, \theta_t) \in (\mathbb{F}_q^*)^{t-1} \\ \theta_2 + \dots + \theta_t = s}} S_{1,(1)}(s). \end{aligned}$$

Since all values $S_{1,(1)}(s)$ are equal to $S_{1,(1)}(1)$ for $s \in \mathbb{F}_q^*$ (Lemma 4 (1)), we obtain

$$\begin{aligned} c_{(1,0,\dots,0)} &= \frac{1}{d^t} (S_{1,(1)}(0) \cdot A_{t-1}(q) + S_{1,(1)}(1)(q^{t-1} - A_{t-1}(q))) \\ &= \frac{1}{d^t} ((q-1) \cdot A_{t-1}(q) - ((q-1)^{t-1} - A_{t-1}(q))) \\ &= \frac{1}{d^t} (q \cdot A_{t-1}(q) - (q-1)^{t-1}) \end{aligned}$$

using Lemma 3 (1) and Lemma 4 (2). Using Lemma 2, we thus get

$$\begin{aligned} c_{(1,0,\dots,0)} &= \frac{1}{d^t} ((q-1)^{t-1} + (-1)^{t-1}(q-1) - (q-1)^{t-1}) \\ &= \frac{1}{d^t} ((-1)^{t-1}(q-1)) = \frac{(-1)^{t-1}}{d^{t-1}}. \end{aligned}$$

Case $i_1 + \dots + i_t = 1 \pmod d$ and there exist two indices $j, k \in \{1, \dots, t\}$ such that $i_j i_k \neq 0$. Let ℓ be the number of indices $j \in \{1, \dots, t\}$ such that $i_j \neq 0$. By assumption, we have $\ell \geq 2$. Without loss of generality, we consider in the following the vector $\mathbf{i} = (i_1, \dots, i_\ell, 0, \dots, 0)$ with $i_1 + \dots + i_\ell = 1 \pmod d$ (the other cases being similar). By the Fourier inversion formula, we have

$$c_{(i_1, \dots, i_\ell, 0, \dots, 0)} = \frac{1}{d^t} \sum_{\theta \in (\mathbb{F}_q^*)^t} \chi(\theta_1 + \dots + \theta_t) \cdot \chi(\theta_1)^{d-i_1} \dots \chi(\theta_\ell)^{d-i_\ell}.$$

As in the previous case, this sum is not a Jacobi sum but can be written as a sum of Jacobi sums depending on the value of the sum $\theta_{\ell+1} + \dots + \theta_t$. Indeed, we have

$$c_{(i_1, \dots, i_\ell, 0, \dots, 0)} = \frac{1}{d^t} \sum_{\theta \in (\mathbb{F}_q^*)^t} \chi(\theta_1 + \dots + \theta_t) \cdot \chi(\theta_1)^{d-i_1} \dots \chi(\theta_\ell)^{d-i_\ell}.$$

This coefficient $c_{(i_1, \dots, i_\ell, 0, \dots, 0)}$ is therefore equal to:

$$\begin{aligned} &\frac{1}{d^t} \sum_{\theta_1, \dots, \theta_\ell \in (\mathbb{F}_q^*)} \sum_{s \in \mathbb{F}_q} \sum_{\substack{(\theta_{\ell+1}, \dots, \theta_t) \in (\mathbb{F}_q^*)^{t-1} \\ \theta_{\ell+1} + \dots + \theta_t = s}} \chi(\theta_1 + \dots + \theta_t) \cdot \chi(\theta_1)^{d-i_1} \dots \chi(\theta_\ell)^{d-i_\ell} \\ &= \frac{1}{d^t} \sum_{s \in \mathbb{F}_q} \sum_{\substack{(\theta_{\ell+1}, \dots, \theta_t) \in (\mathbb{F}_q^*)^{t-1} \\ \theta_{\ell+1} + \dots + \theta_t = s}} S_{\ell, (i_1, \dots, i_\ell)}(s). \end{aligned}$$

Since all values $S_{\ell, (i_1, \dots, i_\ell)}(s)$ are equal to $S_{\ell, (i_1, \dots, i_\ell)}(1)$ for $s \in \mathbb{F}_q^*$ (Lemma 4 (1)), we obtain

$$c_{(i_1, \dots, i_\ell, 0, \dots, 0)} = \frac{1}{d^t} (S_{\ell, (i_1, \dots, i_\ell)}(0) \cdot A_{t-\ell}(q) + S_{\ell, (i_1, \dots, i_\ell)}(1)((q-1)^{t-\ell} - A_{t-\ell}(q))).$$

By the triangle inequality, we get:

$$\begin{aligned}
& |c_{(i_1, \dots, i_\ell, 0, \dots, 0)}| \\
& \geq \frac{1}{d^t} \left| |S_{\ell, (i_1, \dots, i_\ell)}(0)| \cdot A_{t-\ell}(q) - |S_{\ell, (i_1, \dots, i_\ell)}(1)|((q-1)^{t-\ell} - A_{t-\ell}(q)) \right| \\
& = \frac{1}{d^t} \left| q^{(\ell-1)/2}(q-1)A_{t-\ell}(q) - q^{(\ell-1)/2}((q-1)^{t-\ell} - A_{t-\ell}(q)) \right| \\
& = \frac{q^{(\ell-1)/2}}{d^t} |qA_{t-\ell}(q) - (q-1)^{t-\ell}| \\
& = \frac{q^{(\ell-1)/2}}{d^t} |(-1)^{t-\ell}(q-1)| = \frac{q^{(\ell-1)/2}}{d^{t-1}} > \frac{1}{d^{t-1}}.
\end{aligned}$$

using Lemma 3 (2), Lemma 4 (3) and Lemma 2. $\square$

B Complete reconstruction of the polynomial e_0 .

Given an unknown t sparse polynomial e_0 , we assume that we have a set S of pairs $(\mathbf{x}, e_0(\mathbf{x}))$. Let $n' < n$,

$$J = \{(a_1, \dots, a_{n'}) \in \{1, \dots, n\}^{n'} \mid \forall k \in 1, \dots, n' - 1 \ a_k < a_{k+1}\}$$

$\mathbf{j} = (j_1, \dots, j_{n'}) \in J$ and $\mathbf{i} = (i_1, \dots, i_{n'}) \in \{0, \dots, d-1\}^{n'}$. We set $\tilde{X}_{\mathbf{j}}^{\mathbf{i}} = X_{j_1}^{i_1} \dots X_{j_{n'}}^{i_{n'}}$ and $P_{\mathbf{j}\mathbf{i}} = P_{\mathbf{i}}(X_{j_1}, \dots, X_{j_{n'}})$. For all $\mathbf{j} \in J$, e_0 can be written in the form

$$e_0 = \sum_{\mathbf{i} \in \{0, \dots, d-1\}^{n'}} \tilde{X}_{\mathbf{j}}^{\mathbf{i}} P_{\mathbf{j}\mathbf{i}}$$

Let $S_{\mathbf{j}\mathbf{i}}$ be a set of pairs of the form $(\mathbf{x}, P_{\mathbf{j}\mathbf{i}}(\mathbf{x})) \in \mathbb{F}_q^{n-n'} \times \mathbb{F}_q$ and S the set of some evaluations of e_0 . The complete reconstruction algorithm 9 for e_0 takes as input S , n , n' , d and t .


```

1: function  $e_0$ -RECONSTRUCTOR( $S, n, n', d, t$ )
2:    $e_0 \leftarrow [ ]$ 
3:   if  $t = 0$  then
4:     return  $e_0$ 
5:   for  $(i, j) \in \{0, \dots, d-1\}^{n'} \times J$  do
6:     Compute  $S_{ji}$  from  $S$ 
7:     Interpolate  $P_{ji}$  with  $S_{ji}$ 
8:     if  $P_{ji} \neq 0$  then
9:       Update  $e_0$ 
10:       $t \leftarrow t - wt(P_{ji})$ 
11:      Update  $S$ 
12:      Goto line 3
13:   return  $\perp$ 

```

Fig. 9. e_0 -reconstructor.